

Programadores para la Paz

¡La paz se programa desde las regiones!

[Código
que transforma
realidades]



Tecnología
Educación y
Solidaridad



Contenido

02

Presentación

08

Parte 1 Fundamentos para comenzar

11

Parte 2 Instalación y alistamiento técnico

15

Parte 3 Comprender HTML, CSS y estructura web

19

Parte 4 Comprender Node.js y el backend

24

Parte 5 Uso pedagógico y responsable de IA

25

Parte 6 Desarrollo de los contenidos de la capacitación menús semana a semana

88

Parte 7 Guías prácticas complementarias

93

Parte 8 Seguridad, privacidad y buenas prácticas

95

Parte 9 Evaluación y seguimiento

97

Parte 10 Glosario básico del estudiante

99

Parte 11 Recordatorios finales para el estudiante

Presentación

Esta cartilla fue diseñada para acompañarte durante el programa Capacitación en Democracia y Tecnología. Su propósito es servir como una guía práctica y sencilla para que puedas avanzar paso a paso, incluso si no tienes experiencia previa en programación, desarrollo web o uso de herramientas tecnológicas.

No necesitas saber programar antes de comenzar. El programa está organizado para que aprendas de manera progresiva: primero comprenderás el entorno digital y su relación con la democracia; luego conocerás herramientas básicas de trabajo tecnológico; y finalmente aplicarás esos aprendizajes en la construcción de soluciones digitales sencillas, útiles y con sentido social.

A lo largo de esta cartilla encontrarás explicaciones, ejemplos, guías prácticas, recomendaciones, actividades de apoyo, conceptos clave y recordatorios para cada etapa del proceso formativo. Este material no reemplaza al docente ni a las sesiones de clase; funciona como una herramienta de consulta permanente para reforzar lo aprendido y orientar tu práctica.

“ Durante el recorrido tendrás presentes tres ideas centrales: ”



La tecnología no es solo técnica: también tiene impacto social.



Aprender haciendo es la mejor forma de avanzar.



Toda solución digital debe usarse con responsabilidad, cuidado y sentido comunitario.

Cómo usar esta cartilla

Te recomendamos usar esta cartilla de la siguiente manera:

- Lee al inicio la ruta general del programa para entender cómo está organizada la formación.
- Antes de comenzar una semana, revisa el resumen de contenidos y conceptos clave.
- Durante la práctica, usa las guías paso a paso para instalar herramientas, crear archivos y ejecutar comandos.
- Después de cada actividad, vuelve a las secciones de repaso, glosario y cheatsheets.
- Si algo no te sale a la primera, no te frustres. Vuelve a leer, repite el paso y pide apoyo.

Aprender tecnología requiere paciencia. Nadie domina todos los conceptos desde el primer día. Lo importante es avanzar con orden, práctica y constancia.

Propósito general del programa

El programa busca que los estudiantes comprendan la relación entre tecnología, democracia, participación ciudadana, comunicación digital y construcción de comunidades. Al mismo tiempo, promueve el desarrollo de habilidades básicas para crear soluciones digitales sencillas, organizadas y orientadas a resolver necesidades sociales o comunitarias.

Al finalizar el proceso, el estudiante habrá fortalecido capacidades para:

- Comprender el impacto de las tecnologías digitales en la vida social;
- Identificar riesgos y buenas prácticas en entornos digitales;
- Aplicar pensamiento computacional y algoritmia básica;
- Usar Git como herramienta de organización y control de versiones;
- Conocer HTML y CSS para estructurar y presentar contenidos web;
- Iniciar el uso de Node.js para comprender el funcionamiento del backend;
- Organizar datos, rutas y respuestas básicas de una aplicación;
- Aplicar principios iniciales de seguridad digital y protección de información;
- Usar herramientas de automatización con criterio ético y práctico;
- Desarrollar un proyecto integrador con propósito social

Ruta general del programa



El programa se desarrolla en 12 semanas, organizadas en 3 fases progresivas y 6 módulos.

Fase 1. Introducción a la cultura digital, fundamentos técnicos y desarrollo web

Semanas 1 a 3

En esta fase se construye la base. El estudiante entiende cómo funcionan las plataformas digitales, comienza a pensar en pasos lógicos para resolver problemas, conoce herramientas básicas y se aproxima a HTML y Node.js.

Fase 2. Comunicación digital, herramientas tecnológicas y comunidades participativas

Semanas 4 a 7

En esta fase el estudiante conecta lo técnico con la participación ciudadana. Aprende sobre comunicación digital, redes, comunidad, acceso a información, gestión de usuarios y organización de procesos con apoyo tecnológico.

Fase 3. Infraestructura tecnológica, seguridad y desarrollo de plataformas digitales participativas

Semanas 8 a 12

En esta fase el estudiante se aproxima a la operación de sistemas, seguridad digital, despliegue, automatización, metodologías ágiles, bases de datos, autenticación y consolidación de un proyecto integrador.

Módulos del programa

Módulo 1. Tecnologías de la Comunicación y Desarrollo de Aplicaciones Web

Semanas 1 a 3

Módulo 2. Comunicación Digital para la Paz y la Democracia

Semanas 4 a 5

Módulo 3. Construcción de Comunidades Digitales para la Participación Ciudadana Semanas 6 a 7

Módulo 4. Sistemas Operativos de Código Abierto y Seguridad Digital
Semanas 8 a 9

Módulo 5. Desarrollo de Software para la Construcción de Espacios
Digitales Democráticos
Semanas 10 a 11

Módulo 6. Desarrollo de Plataformas Digitales para la Participación
Ciudadana Semana 12

Metodología del programa explicada de forma simple

La metodología del programa se basa en el aprendizaje progresivo, la práctica constante y la relación entre tecnología y ciudadanía. No se espera que el estudiante domine todos los temas desde el inicio. La formación está diseñada para avanzar de manera ordenada, semana a semana.

El proceso se desarrolla a partir de cinco principios:

1. Aprender desde cero

El programa está diseñado para estudiantes que pueden no tener experiencia previa. Por eso, antes de entrar de lleno a temas como Node.js, se explican bases como algoritmia, estructura web, terminal y Git.

2. Aprender paso a paso

Cada semana se apoya en lo aprendido anteriormente. El objetivo no es memorizar, sino comprender y practicar.

3. Combinar teoría y práctica

Los conceptos se explican con ejemplos sencillos y luego se aplican mediante actividades técnicas y reflexivas.

4. Conectar la tecnología con un propósito social

Las herramientas digitales se estudian como medios para informar, organizar, participar, comunicar y construir soluciones útiles para comunidades.

5. Dejar evidencia del aprendizaje

El trabajo no solo se hace: también se registra. Por eso el uso de Git, los repositorios, las bitácoras, los foros y las actividades prácticas son parte importante del proceso.

¿Qué significa la evidencia dual?

Durante el programa se trabaja con una idea muy importante: cada semana debe dejar una evidencia técnica y una evidencia ciudadana o reflexiva.

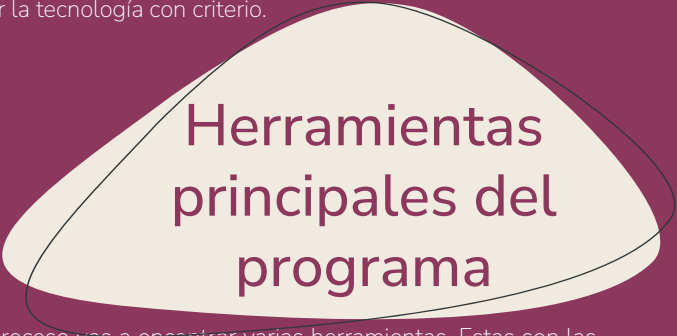
Evidencia técnica

Es el producto que demuestra que hiciste una práctica: un archivo, una página, un formulario, una ruta, un servidor, un flujo, una automatización, una estructura de datos o una funcionalidad.

Evidencia ciudadana o reflexiva

Es el producto que demuestra que entendiste el sentido social o democrático del ejercicio: una bitácora, una reflexión, una guía, un guion, una matriz, un plan corto, un foro o una explicación argumentada.

Esto significa que el programa no forma solamente para escribir código. Forma para usar la tecnología con criterio.



Herramientas principales del programa

A lo largo del proceso vas a encontrar varias herramientas. Estas son las más importantes.

Moodle

Es el aula oficial del curso. Allí encontrarás actividades, foros, quiz, materiales y enlaces.

Google Meet

Se usa para sesiones sincrónicas, acompañamiento y explicación de contenidos.

Git

Sirve para llevar el historial de cambios del proyecto. Te permite registrar avances, ordenar el trabajo y recuperar versiones anteriores.

GITHUB

Es la plataforma donde puedes publicar tu repositorio y compartir tu trabajo.

CURSOR IDE

Es el editor o entorno de trabajo recomendado para escribir y organizar archivos del proyecto. También puede apoyar con inteligencia artificial, siempre bajo verificación.

NODE.JS

Es el entorno que permite ejecutar JavaScript fuera del navegador, especialmente para crear servidores y lógica del backend.



Parte 1

Fundamentos para comenzar

1. Ciudadanía digital en palabras sencillas

Ser ciudadano digital no significa solamente usar internet, redes sociales o aplicaciones. Significa participar en los entornos digitales con responsabilidad, respeto y criterio.

Una persona ejerce ciudadanía digital cuando:

- Respeto a otras personas en línea;
- Piensa antes de publicar o compartir información;
- Verifica las fuentes antes de creer o difundir un contenido;
- Protege sus datos personales y los de los demás;
- Participa con argumentos y sin agresiones;
- Reconoce que lo que ocurre en internet también puede tener consecuencias reales

Ejemplo simple:

Si una persona comparte una noticia falsa sin revisarla, puede dañar a otros. Si una persona trata con respeto a los demás, ayuda a construir convivencia digital.

2. Plataformas digitales y participación

Una plataforma digital es un espacio tecnológico en el que las personas pueden interactuar, compartir información, realizar acciones o acceder a servicios.

Ejemplos:

- una red social
- un formulario de participación



- Una página de información comunitaria
- Una plataforma educativa
- Una aplicación para reportes ciudadanos

Las plataformas no son solo herramientas técnicas. También son espacios donde circulan ideas, decisiones, debates, mensajes y relaciones humanas.

3. Frontend y backend explicados fácil

Frontend

Es la parte visual de una aplicación o página web. Es lo que el usuario ve y toca.

Ejemplos:

- Botones
- Formularios
- Textos
- Menús
- Imágenes
- Páginas

Backend

Es la parte interna que procesa información.

Ejemplos:

- Guardar datos
- Validar usuarios
- Responder solicitudes
- Consultar bases de datos
- Decidir qué información mostrar

Forma fácil de entenderlo

Si una página web fuera un restaurante:

- El frontend sería el salón, la carta y la atención visible;
- El backend sería la cocina y la lógica interna que hace posible el servicio.

4. Pensamiento computacional

Es una forma de pensar problemas paso a paso. No es exclusivo de programadores. También sirve en la vida diaria.

Pensamiento computacional significa:

- Entender un problema
- Dividirlo en partes
- Ordenar acciones
- Tomar decisiones
- Buscar una solución clara

Ejemplo: Problema: compartir una noticia en internet.

Pasos posibles: 1. leer la noticia completa; 2. revisar la fuente; 3. comparar con otra fuente; 4. validar la fecha; 5. decidir si es confiable; 6. compartir o no compartir.

Eso ya es una forma básica de algoritmo.

5. ¿Qué es un algoritmo?

Un algoritmo es una lista ordenada de pasos para resolver una tarea o problema.

Ejemplo cotidiano

Algoritmo para preparar una bebida caliente: 1. buscar una taza; 2. calentar agua; 3. poner el contenido en la taza; 4. agregar el agua; 5. mezclar; 6. servir.

Ejemplo digital

Algoritmo para verificar una información: 1. leer el contenido; 2. revisar quién lo publicó; 3. validar la fecha; 4. buscar confirmación en otro lugar; 5. evaluar si es confiable; 6. compartir o no compartir.

6. Entrada, proceso y salida

Esta es una forma sencilla de organizar la lógica de un problema.

Entrada

Es la información que llega.

Proceso

Es lo que haces con esa información.

Salida

Es el resultado final.

Ejemplo: Problema: verificar si una publicación se puede compartir.

Entrada:

- Publicación
- Fuente
- Fecha

Proceso:

- Revisar la fuente
- Comparar con otra fuente
- Decidir si es confiable

Salida:

- Compartir
- No compartir la publicación y alistamiento técnico

Parte 2

Instalación y alistamiento técnico

7. Instalación de Git paso a paso

Git es una herramienta de control de versiones. Sirve para registrar cambios y llevar orden en el trabajo.

¿Para qué sirve Git?

- Guardar el historial de cambios;
- Registrar avances
- Volver a una versión anterior;
- Trabajar con mayor orden;
- Compartir el proyecto con otros.

Antes de instalar

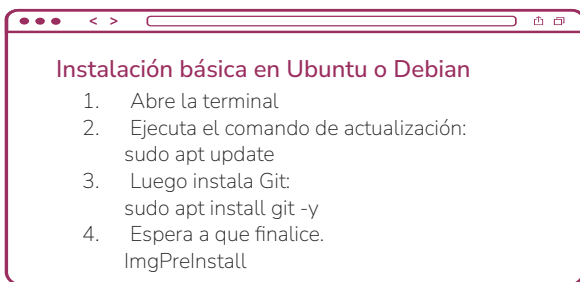
Revisa qué sistema operativo tienes:

- Windows
- Linux
- macOS

Instalación básica en Windows

1. Abre el navegador
2. Busca el instalador oficial de Git
3. Descarga la versión para Windows
4. Ejecuta el instalador
5. Deja las opciones por defecto, a menos que el docente te indique otra cosa
6. Termina la instalación
7. Abre Git Bash o la terminal





Verificar la instalación
Escribe en la terminal:
`git --version`

Si Git quedó bien instalado, verás una respuesta con el número de versión.

Configuración inicial de Git
Después de instalar, configura tu nombre y correo:

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tucorreo@ejemplo.com"
```

Verificar la configuración
`git config --list`

Qué debes recordar

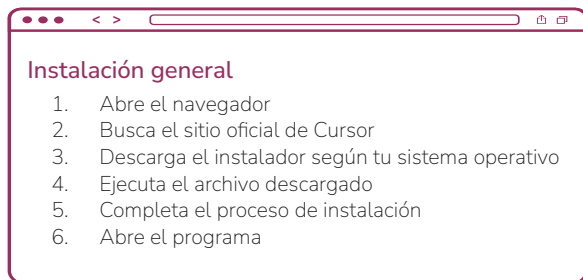
- Instalar Git no es lo mismo que crear una cuenta en GitHub
- Git funciona en tu computador
- GitHub es una plataforma para publicar repositorios

8. Instalación de Cursor IDE paso a paso

Cursor es un editor de código. Allí puedes abrir carpetas, crear archivos, editar proyectos y apoyarte con funciones inteligentes.

¿Para qué sirve un editor de código?

- Crear archivos
- Escribir código
- Ordenar carpetas
- Editar texto
- Revisar errores
- Trabajar con proyectos



Primeros pasos en Cursor

Cuando abras Cursor por primera vez: 1. ubica el menú para abrir carpeta; 2. crea o selecciona una carpeta de trabajo; 3. revisa el panel lateral de archivos; 4. crea un archivo nuevo; 5. guarda los cambios.

Operaciones básicas en Cursor

- Open Folder: abrir carpeta de trabajo
- New File: crear archivo
- Save: guardar archivo
- Explorer: ver estructura de carpetas
- Terminal: abrir terminal integrada.

Recomendación: No uses la ayuda de IA para copiar sin entender. Usa la IA para:

- Pedir explicaciones
- Aclarar errores
- Entender un ejemplo
- Revisar una idea
- Mejorar una redacción técnica

Siempre debes verificar lo que te entregue

9. Instalación de Node.js paso a paso

Node.js permite ejecutar JavaScript fuera del navegador. Esto es útil para crear servidores y trabajar la parte interna de una aplicación.

¿Para qué sirve Node.js?

- Crear servidores
- Ejecutar archivos JavaScript en tu equipo
- Trabajar con paquetes
- Desarrollar backend
- Usar herramientas del ecosistema JavaScript.

Instalación básica en Windows

1. Abre el navegador.
2. Busca el sitio oficial de Node.js.
3. Descarga la versión recomendada o estable.
4. Ejecuta el instalador.
5. Avanza con la configuración por defecto.
6. Finaliza la instalación.
7. Abre una terminal nueva.

Instalación básica en Ubuntu o Debian

Dependiendo del entorno, el docente puede indicar la ruta más adecuada. Una forma común es usar el gestor de paquetes o un repositorio recomendado.

Si ya está disponible en tu entorno:

```
sudo apt update
```

```
sudo apt install nodejs npm -y
```

Verificar la instalación

Revisa primero Node.js:

```
node -v
```

Luego revisa npm:

```
npm -v
```

Si ambos responden con una versión, la instalación quedó correcta.

Qué es npm

npm es el administrador de paquetes de Node.js. Sirve para instalar librerías o dependencias que ayudan en el desarrollo.

10. Crear una cuenta y un repositorio en GitHub

GitHub es el lugar donde puedes publicar tu proyecto y compartirlo.

Pasos generales

1. Entra a GitHub.
2. Crea una cuenta.
3. Inicia sesión.
4. Busca la opción para crear un repositorio nuevo.
5. Ponle un nombre al repositorio.
6. Crea el repositorio.

Nombre sugerido del repositorio

En varias actividades del programa se trabaja con un repositorio llamado: programadores-para-la-paz

Relación entre Git y GitHub

- Git registra los cambios.
- GitHub publica y aloja el repositorio.

11. Primeros comandos de terminal

La terminal es una herramienta para ejecutar instrucciones escritas.

¿Por qué es importante?

Porque muchas tareas técnicas se hacen desde allí:

- Navegar carpetas
- Ejecutar programas
- Instalar paquetes
- Lanzar servidores
- Usar Git

Comandos básicos. Ver dónde estás:

En Linux o macOS:

```
pwd
```

Volver a la carpeta anterior

```
cd ..
```

Listar archivos y carpetas

```
ls
```

Crear una carpeta

```
mkdir mi-carpeta
```

Entrar a una carpeta

```
cd nombre-de-la-carpeta
```

Crear un archivo vacío en Linux o macOS

```
touch archivo.txt
```

Idea importante: La terminal no se aprende en un solo día. Se aprende con práctica.

Parte 3

Comprender HTML, CSS y estructura web

12. ¿Qué es HTML?

HTML significa HyperText Markup Language. Es el lenguaje que da estructura a una página web.

HTML no es un lenguaje de programación en el sentido clásico. No toma decisiones complejas por sí mismo. Su función principal es organizar el contenido.

¿Qué organiza HTML?

- Títulos
- Párrafos
- Listas
- Enlaces
- Imágenes
- Formularios
- Secciones

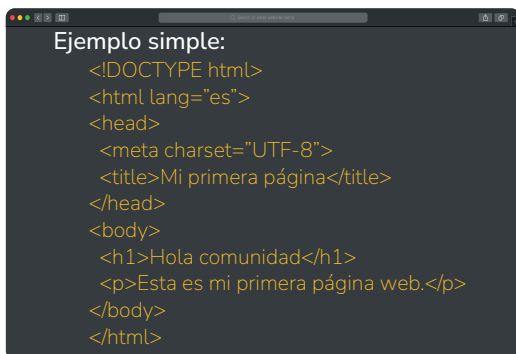
Forma fácil de entenderlo

Si una página web fuera una casa:

- HTML sería la estructura de la casa
- CSS sería la pintura y la apariencia
- JavaScript daría comportamiento
- Node.js ayudaría a resolver la parte del servidor



13. Estructura básica de un archivo HTML



Explicación de cada parte

- `<!DOCTYPE html>`: indica que el documento es HTML.
- `<html>`: contiene todo el documento.
- `<head>`: guarda información general del documento.
- `<meta charset="UTF-8">`: ayuda a manejar caracteres correctamente.
- `<title>`: define el título de la pestaña del navegador.
- `<body>`: contiene lo que se ve en la página.
- `<h1>`: título principal.
- `<p>`: párrafo.

14. Etiquetas HTML básicas

Títulos

```
<h1>Título principal</h1>
<h2>Subtítulo</h2>
<h3>Otro nivel de título</h3>
```

Párrafos

```
<p>Este es un párrafo.</p>
```

Listas

```
<ul>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
</ul>
```

Enlaces

```
<a href="https://ejemplo.com">Visitar
sitio</a>
```

Imágenes

```

```

Secciones

```
<section>
  <h2>Noticias</h2>
  <p>Contenido de la sección.</p>
</section>
```

15. ¿Qué es HTML semántico?

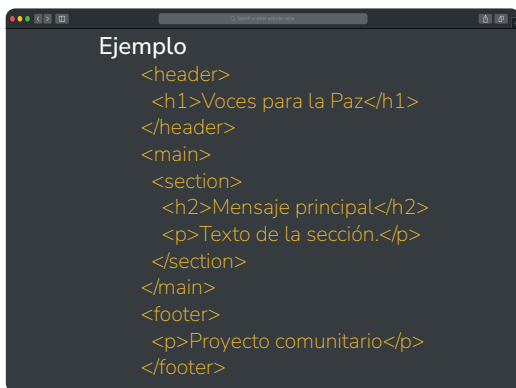
Significa usar etiquetas que tienen sentido según el contenido.

Por ejemplo:

- Header para la cabecera
- Main para el contenido principal
- Section para una sección
- Article para un contenido independiente
- Footer para el cierre o pie de página

¿Por qué importa? Porque ayuda a:

- Organizar mejor el contenido
- Facilitar la lectura del código
- Mejorar accesibilidad
- Dar más claridad a la estructura

A screenshot of a code editor window with a dark background. The title bar at the top says "Ejemplo". The code is written in a light yellow font and shows a basic HTML structure with header, main, section, and footer tags.

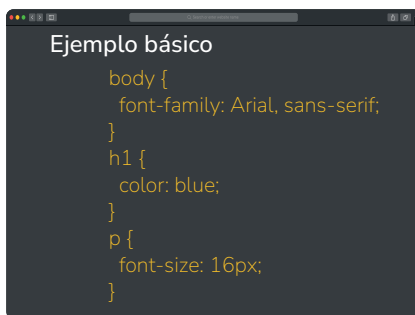
```
<header>
  <h1>Voces para la Paz</h1>
</header>
<main>
  <section>
    <h2>Mensaje principal</h2>
    <p>Texto de la sección.</p>
  </section>
</main>
<footer>
  <p>Proyecto comunitario</p>
</footer>
```

16. ¿Qué es CSS?

CSS significa Cascading Style Sheets. Sirve para dar estilo visual a una página.

Con CSS puedes cambiar:

- Color del texto;
- Tamaño de letra;
- Espacios;
- Bordes;
- Alineación;
- Tamaño de elementos.

A screenshot of a code editor window with a dark background. The title bar at the top says "Ejemplo básico". The code is written in a light yellow font and shows CSS rules for the body, h1, and p elements.

```
body {
  font-family: Arial, sans-serif;
}
h1 {
  color: blue;
}
p {
  font-size: 16px;
}
```

Relación entre HTML y CSS

- HTML pone la estructura.
- CSS mejora la presentación.

17. Formularios accesibles

Un formulario es una estructura que permite que una persona escriba y envíe información.

Buenas prácticas

- Usar etiquetas claras
- Explicar para qué sirve cada campo
- No pedir datos innecesarios
- Usar mensajes comprensibles
- Organizar bien los campos

Ejemplo simple

```
<form>
  <label for="nombre">Nombre</label>
  <input type="text" id="nombre" name="nombre">
  <label for="mensaje">Mensaje</label>
  <textarea id="mensaje" name="mensaje"></textarea>

  <button type="submit">Enviar</button>
</form>
```

¿Qué hace label?

Ayuda a que la persona entienda qué debe escribir en el campo.

18. Cheatsheet rápido de HTML

Estructura mínima

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Título</title>
</head>
<body>
</body>
</html>
```

Etiquetas más usadas

- H1 a h6: títulos
- P: párrafo
- A: enlace
- Img: imagen
- Ul y li: lista
- Section: sección
- Form: formulario
- Label: etiqueta de campo
- Input: entrada
- Textarea: caja de texto amplia
- Button: botón

Recomendaciones rápidas

- Usa un solo h1 principal
- Escribe textos claros
- Da nombres útiles a las secciones
- Usa label en formularios
- Evita pedir datos de más
- Mantén el código ordenado

Parte 4

Comprender Node.js y el backend

19. ¿Qué es Node.js en términos generales?

Node.js es un entorno de ejecución que permite usar JavaScript fuera del navegador. Con él se pueden crear servidores, organizar rutas, recibir solicitudes y construir la parte interna de una aplicación."

Esto significa que JavaScript no solo sirve para hacer cosas en la parte visible de una página. También puede servir para trabajar del lado del servidor.

¿Para qué sirve Node.js?

- Crear servidores
- Recibir solicitudes
- Responder información
- Conectar con bases de datos
- Trabajar con APIs
- Organizar la lógica interna de una aplicación.
-

Idea simple

Con HTML construyes la parte visible. Con Node.js puedes construir la parte que responde, procesa y organiza datos.

20. ¿Qué es un servidor?



Un servidor es un programa o equipo que responde solicitudes.

Ejemplo cotidiano: Cuando abres una página web, tu navegador hace una solicitud. El servidor responde enviando información.

En Node.js

Puedes crear un servidor que escuche en un puerto y responda mensajes, datos o contenido.

21. Primer archivo con Node.js

Un archivo JavaScript simple:
`console.log('Hola desde Node.js');`

Guárdalo como `app.js`.

Para ejecutarlo:
`node app.js`

Si todo sale bien, verás en la terminal:
Hola desde Node.js

22. Crear un proyecto Node.js

Paso 1. Crear una carpeta

```
mkdir mi-proyecto-node
cd mi-proyecto-node
```

Esto crea un archivo llamado `package.json`.

¿Qué es `package.json`?

Es un archivo que guarda información del proyecto y sus dependencias.

Paso 2. Inicializar el proyecto

```
npm init -y
```

23. Instalar Express

Express es una librería muy usada en Node.js para crear servidores y rutas de manera más sencilla.

Instalar Express: `npm install express`

Cuando se instala, aparece una carpeta `node_modules` y se actualiza el archivo `package.json`.

24. Primer servidor con Express

Crea un archivo llamado `server.js` con este contenido:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Servidor activo');
});

app.listen(3000, () => {
  console.log('Servidor ejecutándose en puerto 3000');
});
```

Ejecutar el servidor

```
node server.js
```

Probar en el navegador

Abre:

```
http://localhost:3000
```

Si todo está bien, verás:

Servidor activo

25. ¿Qué es una ruta?

Una ruta define cómo responde el servidor a una dirección específica.

Ejemplo

Si visitas:

/

/saludo

/mensaje/juan

el servidor puede responder distinto en cada caso.

Ejemplo práctico

```
app.get('/saludo', (req, res) => {  
  res.send('Hola comunidad');  
});
```

Esto significa que cuando alguien entre a /saludo, el servidor responderá con Hola comunidad.

26. GET y POST de manera introductoria

GET

Se usa normalmente para consultar o pedir información.

Ejemplo:

- Ver una lista de mensajes
- Leer una noticia
- Consultar un reporte

POST

Se usa normalmente para enviar o registrar información.

Ejemplo:

- Enviar un formulario
- Registrar un reporte
- Guardar una solicitud

27. JSON explicado fácil

JSON es una forma ordenada de representar información en formato texto.

Ejemplo

```
{
  "nombre": "Ana",
  "mensaje": "Hola comunidad"
}
```

Se usa mucho para:

- enviar datos
- recibir respuestas
- trabajar entre frontend y backend
- compartir información entre sistemas

28. APIs en palabras simples

Una API es una forma organizada de permitir que sistemas o aplicaciones se comuniquen.

Ejemplo sencillo: Una aplicación pide datos a otra. La API define cómo se hace esa solicitud y cómo llega la respuesta.

¿Para qué puede servir una API en contextos comunitarios?

- Consultar información pública;
- Registrar reportes;
- Compartir alertas;
- Mostrar datos actualizados;
- Conectar una interfaz con un servidor.

29. Cheatsheet rápido de Node.js y Express

Ejecutar un archivo

```
node app.js
```

Crear proyecto

```
npm init -y
```

Instalar Express

```
npm install express
```

Servidor básico

```
const express = require('express');
const app = express();
```

```
app.get('/', (req, res) => {
  res.send('Hola');
});
app.listen(3000, () => {
  console.log('Servidor ejecutándose');
});
```

Ruta GET

```
app.get('/saludo', (req, res) => {
  res.send('Hola comunidad');
});
```

Ruta con parámetro

```
app.get('/mensaje/nombre', (req, res) => {
  const nombre = req.params.nombre;
  res.send('Hola ' + nombre);
});
```

Habilitar JSON

```
app.use(express.json());
```

Ruta POST

```
app.post('/reporte', (req, res) => {  
  const mensaje = req.body.mensaje;  
  res.json({  
    estado: 'Reporte recibido',  
    mensaje: mensaje  
  });  
});
```

30. Cheatsheet rápido de Git

Ver versión

```
git --version
```

Configurar nombre

```
git config --global user.name "Tu Nombre"
```

Configurar correo

```
git config --global user.email "tucorreo@ejemplo.com"
```

Iniciar repositorio

```
git init
```

Ver estado

```
git status
```

Agregar cambios

```
git add .
```

Crear commit

```
git commit -m "Mensaje claro"
```

Cambiar nombre de rama principal

```
git branch -M main
```

Conectar repositorio remoto

```
git remote add origin URL_DEL_REPOSITORIO
```

Publicar cambios

```
git push -u origin main
```

Idea clave

- git add . prepara cambios.
- git commit registra una versión.
- git push envía los cambios a GitHub.

Parte 5

Uso pedagógico y responsable de IA

31. Cómo usar IA sin depender de ella

Durante el programa puedes usar herramientas como Cursor, Gemini u otra IA de apoyo. Pero debes usarlas con criterio.

Sí se recomienda usar IA para:

- Pedir explicaciones;
- Entender conceptos difíciles;
- Revisar un error;
- Pedir ejemplos sencillos;
- Mejorar redacciones;
- Comparar soluciones.

No se recomienda usar IA para:

- Copiar sin leer;
- Entregar respuestas que no entiendes;
- Dejar que piense por ti todo el tiempo;
- Usar información sin verificar;
- Enviar datos sensibles.

Regla de oro

La IA puede apoyar, pero la responsabilidad final sigue siendo tuya.



Parte 6

Desarrollo de los contenidos de la capacitación semana a semana

1

SEMANA

2

SEMANA

3

SEMANA

4

SEMANA

5

SEMANA

6

SEMANA

7

SEMANA

8

SEMANA

9

SEMANA

10

SEMANA

11

SEMANA

12

SEMANA



Semana 1

Ciudadanía digital, tecnología y comprensión del entorno digital

Bienvenida a la primera semana

Esta primera semana tiene un propósito muy importante: ayudarte a entender que la tecnología no empieza en el código, sino en la vida real.

Antes de crear una página web, usar Git o instalar Node.js, necesitas comprender el entorno digital en el que vivimos. Todos los días usamos redes sociales, plataformas educativas, mensajería, videos, buscadores y aplicaciones. Sin embargo, usar una herramienta no significa necesariamente entenderla.

Por eso, esta semana es una base. Aquí comenzarás a mirar la tecnología no solo como algo técnico, sino como algo que influye en la comunicación, en la participación ciudadana, en la información que recibimos y en la manera en que convivimos.

¿Por qué estudiar tecnología desde la democracia?

Muchas veces se cree que la tecnología solo le sirve a las personas programadoras o a quienes trabajan en sistemas. Pero eso no es cierto.

La tecnología también influye en:

- cómo se informa una comunidad;
 - cómo circulan las ideas;
- cómo se comparten opiniones;



- Cómo se participa en causas sociales;
- Cómo se organizan grupos de personas;
- Cómo se toma posición frente a un tema público.

En otras palabras, la tecnología también tiene una dimensión social. Una plataforma no es solo un conjunto de botones. También es un espacio donde circulan mensajes, conflictos, acuerdos, campañas, debates y decisiones.

Por eso esta capacitación une dos cosas: democracia y tecnología.

¿Qué es la ciudadanía digital?

La ciudadanía digital es la forma en que una persona se comporta, participa y toma decisiones dentro del entorno digital. No se trata solamente de tener internet, abrir una cuenta o saber usar una aplicación. Se trata de comprender que también en internet existen responsabilidades.

Una persona ejerce ciudadanía digital cuando:

- Usa la tecnología con responsabilidad;
- Respeta a otras personas en línea;
- Evita compartir información falsa;
- Protege sus datos;
- Participa con argumentos;
- Cuida el efecto de lo que publica;
- Entiende que internet también tiene consecuencias reales.

La ciudadanía digital no es un tema secundario. Es una forma de convivencia.

Ciudadanía digital no es solo "portarse bien"

A veces se explica este concepto de forma muy simple, como si se tratara solamente de "no pelear en redes". Pero va mucho más allá.

La ciudadanía digital implica aprender a estar en internet con criterio.

Eso significa:

- Saber identificar riesgos;
- Reconocer manipulación;
- Entender que una publicación puede afectar a una comunidad;
- Saber que una opinión también tiene responsabilidad;
- Entender que una herramienta digital puede usarse para construir o para dañar.

Por eso, en este curso, ciudadanía digital significa participación responsable.

Derechos y deberes en internet

Así como en la vida cotidiana hay derechos y deberes, en los entornos digitales también los hay.

- Entre los derechos más importantes están:
- El derecho a expresarse;
- El derecho a informarse;
- El derecho a la privacidad;
- El derecho a participar;
- El derecho a acceder a herramientas de aprendizaje.

Pero junto a esos derechos existen deberes.

Por ejemplo:

- No difundir información dañina sin verificar;
- No atacar a otras personas;
- No exponer datos ajenos;
- No manipular contenido;
- No usar la tecnología para humillar o excluir.

Esto es importante porque internet no está separado de la vida real. Lo que sucede en línea puede afectar la reputación, las emociones, la seguridad y la convivencia de las personas.

La convivencia digital

La convivencia digital es la capacidad de compartir espacios en internet con respeto, responsabilidad y cuidado. Convivir bien en internet no significa pensar igual. Significa saber participar sin destruir a quien piensa distinto.

Esto implica:

- Leer antes de reaccionar;
- No responder con agresión automática;
- No compartir rumores;
- No burlarse del error de otro;
- No usar la exposición pública como castigo.

Muchas violencias actuales pasan por entornos digitales. Por eso la convivencia en internet es tan importante como la convivencia presencial.

Durante esta semana también debes empezar a reconocer que internet tiene oportunidades, pero también riesgos.

Algunos riesgos comunes son:

- Desinformación;
- Suplantación;
- Manipulación emocional;
- Agresión verbal;
- Exposición de datos;
- Engaños;
- Dependencia excesiva de la reacción inmediata.

No se trata de tener miedo a internet. Se trata de usarlo con criterio.

Plataformas digitales: más que herramientas

Una plataforma digital es un espacio tecnológico donde las personas interactúan, publican, responden, consultan información o realizan acciones.

Ejemplos de plataformas digitales son:

- Redes sociales;
- Plataformas educativas;
- Aplicaciones de mensajería;
- Páginas web;
- Formularios de reporte;
- Sistemas de inscripción;
- Tableros de seguimiento.

Pero aquí lo importante no es solamente reconocerlas. Lo importante es comprender que estas plataformas organizan nuestra experiencia.

Una plataforma decide muchas veces:

- Qué te muestra primero;
- Qué contenido se vuelve visible;
- Qué mensajes aparecen más;
- Qué se repite con frecuencia;
- Qué tan fácil es participar;
- Cómo se organiza la información.

Eso significa que las plataformas influyen en la forma en que percibimos la realidad.

El impacto social de las plataformas

Cuando una persona entra a una red social, no ve “todo” lo que existe. Ve una parte seleccionada.

Esa selección influye en:

- Lo que cree importante;
- Lo que considera urgente;
- Lo que piensa que todo el mundo está diciendo;
- La emoción con la que responde a un tema.

Por eso es tan importante que el estudiante entienda desde el comienzo que una plataforma no es neutral. Tiene diseño, lógica, prioridades y efectos.

Frontend y backend explicados de forma simple

Esta semana también introduce dos conceptos fundamentales para todo el curso: frontend y backend.

¿Qué es el frontend?

El frontend es la parte visible de una aplicación o página web.

Es todo lo que una persona ve y usa directamente:

- Botones;
- Textos;
- Menús;
- Imágenes;
- Formularios;
- Colores;
- Secciones.

Cuando una persona entra a una página y ve el contenido, está viendo el frontend.

¿Qué es el backend?

El backend es la parte interna que hace funcionar la lógica del sistema.

Allí ocurren procesos como:

- Guardar datos;
- Recibir solicitudes;
- Validar usuarios;
- Responder información;
- Conectar con bases de datos;
- Decidir qué mostrar.

El backend no siempre se ve, pero es esencial.

Una forma fácil de recordarlo: Imagina una biblioteca digital.

El frontend sería lo que ve la persona:

- El buscador;
- El botón para consultar;
- La lista de resultados;
- La pantalla de acceso.

El backend sería todo lo que pasa detrás:

- La búsqueda de datos;
- La validación del usuario;
- La consulta al sistema;
- La respuesta organizada.

Pensamiento computacional: aprender a ordenar problemas

Otro tema esencial de esta semana es el pensamiento computacional. Este concepto puede sonar técnico, pero en realidad es muy cercano a la vida diaria. Pensar computacionalmente significa aprender a organizar un problema con claridad. No se trata de pensar como una máquina. Se trata de pensar con orden.

Cuatro habilidades del pensamiento computacional

1. Descomponer

Consiste en dividir un problema grande en partes pequeñas.

Por ejemplo, si quieres crear una plataforma para recibir reportes comunitarios, no empiezas haciendo todo de una vez.

Primero lo divides:

- Qué información se debe reportar;
- Quién la enviará;
- Cómo se recibirá;
- Dónde se guardará;
- Cómo se responderá.

2. Reconocer patrones

Significa identificar si algo se parece a otra situación ya conocida.

Por ejemplo, si ya entiendes cómo funciona un formulario de contacto, luego te será más fácil entender un formulario de reporte.

3. Abstractar

Significa quedarte con lo esencial y no perderte en detalles que todavía no necesitas. Esto es muy importante cuando empiezas. No debes querer entenderlo todo al mismo tiempo.

4. Diseñar pasos

Es construir una secuencia clara para resolver una tarea. Y aquí aparece el siguiente concepto.

Algoritmos: pasos claros para resolver una tarea

Un algoritmo es una secuencia ordenada de pasos para llegar a un resultado. No tienes que estar programando para usar algoritmos. También los usas en la vida diaria.

Por ejemplo, si quieres compartir una información de forma responsable, podrías seguir este algoritmo:

1. Leer la publicación completa;
2. Revisar quién la publicó;
3. Confirmar la fecha;
4. Buscar otra fuente;
5. Evaluar si es confiable;
6. Compartir o no compartir.

Eso ya es una forma de algoritmo.

Entrada, proceso y salida. Esta es otra idea útil de la semana.

Entrada

Es la información que llega.

Proceso

Es lo que haces con esa información.

Salida

Es el resultado final.

Por ejemplo:

Entrada:

una publicación de internet.

Proceso:

leer, revisar fuente, contrastar, decidir.

Salida:

compartir o no compartir.

Esta estructura te ayudará mucho más adelante cuando trabajes con formularios, rutas, datos y programación.

Primer contacto con las herramientas técnicas

Aunque esta semana es principalmente conceptual, también te ubica frente al entorno técnico.

Aquí conocerás por primera vez palabras como:

- Editor de código;
- Repositorio;
- Git;
- Terminal;
- Proyecto;
- Archivo;
- Carpeta.

No se espera que ya domines todo esto. Lo importante es empezar a familiarizarte con esos nombres y entender que forman parte del camino de aprendizaje.

Lo que debes llevarte de esta semana

Al terminar esta semana, lo más importante es que entiendas cinco ideas:

Primera: la tecnología tiene impacto social.

Segunda: internet requiere responsabilidad.

Tercera: una plataforma no solo muestra cosas, también organiza lo que vemos.

Cuarta: frontend y backend son dos partes de una misma solución.

Quinta: programar empieza pensando con orden.

Tip de la semana

Si algo te parece nuevo o difícil, no lo rechaces de inmediato. Esta primera semana está diseñada para darte base, no para exigirte perfección.

Retos frecuentes y cómo resolverlos

Pensar que como todavía no has escrito código, entonces todavía no has empezado a aprender tecnología. Eso es falso.

Ya estás aprendiendo tecnología cuando entiendes cómo impacta la información, cómo se organizan las plataformas y cómo se resuelve un problema paso a paso.

Semana 2

Noticias falsas, verificación responsable, opinión pública y primeros pasos técnicos con terminal, HTML y CSS

El propósito de esta semana

En esta semana darás un paso muy importante. Ya no solo entenderás que internet tiene impacto social. Ahora comenzarás a analizar cómo circula la información y por qué muchas veces lo que se comparte más no es necesariamente lo más verdadero. También empezarás a acercarte al entorno técnico con mayor claridad. Verás terminal, HTML y CSS como herramientas iniciales para construir y organizar información digital.

Esta semana une dos dimensiones:

- la comprensión crítica de la información;
- el inicio de la construcción básica de interfaces.

Noticias falsas: qué son y por qué importan

Una noticia falsa es un contenido que presenta información engañosa, inventada, manipulada o descontextualizada como si fuera real. No siempre una noticia falsa es completamente inventada. A veces mezcla elementos verdaderos con interpretaciones engañosas.



A veces usa una imagen real
para contar algo falso.
En otros casos reutiliza una
noticia vieja como si fuera actual.

Lo importante es entender que una noticia falsa no solo es “información incorrecta”. Es un contenido que puede producir consecuencias reales.

Formas comunes de desinformación

La desinformación puede aparecer de varias maneras.

Información inventada

Es completamente falsa y se construye desde cero.

Información descontextualizada

Usa un hecho real, pero lo presenta en otro contexto para cambiar su sentido.

Información vieja presentada como nueva

Esto ocurre mucho cuando circulan publicaciones antiguas en momentos de tensión.

Imágenes o videos fuera de contexto

Una imagen real puede utilizarse para respaldar una historia falsa.

Titulares manipulados

El texto exagera o cambia el sentido del contenido para generar reacción.

¿Por qué las noticias falsas circulan tan rápido?

Las noticias falsas suelen circular rápido porque apelan a emociones fuertes. Cuando algo produce miedo, rabia, sorpresa o indignación, muchas personas reaccionan antes de pensar. Eso hace que compartan primero y verifiquen después.

Además, estas publicaciones suelen tener características que facilitan su expansión:

- Títulos impactantes;
- Lenguaje exagerado;
- Imágenes llamativas;
- Promesas de “verdades ocultas”;
- Llamados urgentes a compartir;
- Mensajes que hacen sentir que hay que reaccionar de inmediato.

La emoción como acelerador

Uno de los aprendizajes más importantes de esta semana es este: cuando un contenido te hace sentir una emoción muy intensa, debes verificar; más, no menos. Muchas veces la manipulación digital se apoya en eso. No busca que pienses con calma. Busca que reacciones rápido. Por eso, una persona formada en ciudadanía digital debe aprender a detectar cuándo una emoción está nublando el juicio.

¿Por qué la desinformación es peligrosa?

La desinformación no es un problema pequeño.

Puede afectar:

- La reputación de una persona;
- La tranquilidad de una comunidad;

- La calidad del debate público;
- La confianza en instituciones;
- La toma de decisiones;
- La participación democrática.

En una comunidad, una noticia falsa puede hacer que la gente desconfíe, se altere o actúe sobre información equivocada.

En contextos democráticos, la desinformación puede deformar el debate. Si las personas discuten desde mentiras, el proceso de participación pierde calidad.

Señales de alerta para detectar contenido dudoso

Hay varias señales que te deben hacer detener antes de compartir.

Título excesivamente escandaloso

Si parece diseñado para provocar impacto antes que comprensión, desconfía.

Fuente desconocida o ausente

Si no sabes quién publicó el contenido o no aparece una fuente clara, debes revisar más.

Fecha poco visible o inexistente

Sin fecha, no puedes saber si el contenido sigue vigente.

Lenguaje agresivo o exagerado

Muchas veces la manipulación usa expresiones exageradas para empujar la reacción.

Falta de datos verificables

Si no hay nombres, cifras claras, documentos o referencias, el contenido pierde credibilidad.

Imagen sospechosa

Si la imagen parece muy impactante pero no tiene contexto, revisa antes de creer.

Verificación responsable: qué significa realmente

Verificar información no es desconfiar de todo. Tampoco es convertirse en investigador profesional para cada publicación. Verificar; significa revisar lo suficiente antes de creer o compartir. Es una práctica de responsabilidad.

¿Por qué verificar es una acción ética?

Porque cuando compartes algo en internet, no solo estás repitiendo un mensaje. También puedes estar moviendo miedo, esperanza, rabia, rechazo o apoyo.

Compartir no es una acción neutra.

Por eso verificar; es también una forma de cuidado hacia otras personas.

1. Leer más allá del titular

Mucha gente comparte solo por el título. Ese es uno de los errores más frecuentes. Hay que leer el contenido completo.

2. Revisar quién publica

No es lo mismo una fuente reconocida, una institución, un medio serio o un documento verificable, que una cuenta anónima o una página improvisada.

3. Confirmar la fecha

Una noticia vieja puede ser técnicamente real, pero estar siendo usada de forma engañosa.

4. Buscar una segunda fuente

Si la información solo aparece en un lugar dudoso, no merece confianza.

5. Revisar imágenes y contexto

A veces el engaño está en la forma en que se usa una imagen.

6. Pensar a quién puede afectar

Antes de compartir, pregúntate: si esto fuera falso, ¿qué daño podría causar?

¿Qué hacer cuando no puedes verificar?

No compartir. Esta es una de las decisiones más inteligentes del entorno digital. No todo lo que llega debe circular.

Plataformas digitales y opinión pública

Otro contenido central de esta semana es entender cómo las plataformas influyen en la opinión pública.

La opinión pública es el conjunto de percepciones, ideas y debates que circulan sobre asuntos importantes. Hoy, buena parte de esa conversación ocurre en internet.

Las plataformas no son neutrales

Una plataforma organiza lo que ves. Eso significa que decide, de distintas maneras, qué contenido gana más visibilidad, qué se repite, qué aparece primero y qué pasa desapercibido. Esto influye en cómo interpretas el mundo.

Riesgos de no entender el funcionamiento de una plataforma

Si no entiendes cómo funcionan las plataformas, puedes caer en varios errores:

- Creer que lo más viral es lo más verdadero;
- Pensar que todo el mundo opina igual porque ves mucho de lo mismo;
- Reaccionar sin contraste;
- Confundir repetición con verdad;
- Asumir que un contenido visible es automáticamente confiable.

La relación entre opinión pública y democracia

La democracia necesita información, criterio y debate.

Si una plataforma favorece el ruido, la reacción rápida o la polarización, las personas deben aprender a participar con más conciencia.

Por eso esta semana no solo enseña a verificar. También enseña a pensar cómo influye el entorno digital en la construcción de opinión.

La terminal: primer contacto real con el entorno técnico

En la parte técnica, esta semana introduces la terminal. La terminal es una herramienta que permite dar instrucciones escritas al sistema. Al principio puede parecer extraña porque no funciona como una aplicación visual tradicional. Sin embargo, es una herramienta muy útil.

¿Para qué sirve la terminal?

La terminal te permite:

- Moverte entre carpetas;
- Crear estructuras de trabajo;
- Ejecutar programas;
- Abrir proyectos;
- Usar Git;
- Correr aplicaciones con Node.js;
- Instalar paquetes.

La terminal te ayuda a trabajar con mayor orden y precisión.

¿Por qué el curso introduce la terminal temprano?

Porque trabajar solo haciendo clic a veces oculta cómo funciona realmente el entorno.

La terminal te da una relación más clara con el sistema.

Además, muchas tareas del desarrollo web real se hacen allí.

HTML: estructura de la información

Esta semana también introduces HTML. HTML es el lenguaje **que** organiza el contenido de una página web. Sirve para dar estructura.

Eso significa que ayuda a definir:

- Títulos;
- Párrafos;
- Listas;
- Imágenes;
- Formularios;
- Enlaces;
- Secciones.

HTML no es decoración

Una idea importante para el estudiante es que HTML no está para “llenar” la pantalla. Está para organizar el contenido de manera comprensible. Un buen HTML ayuda a que una persona entienda qué está viendo, dónde está la información importante y qué debe hacer.

La lógica de una estructura clara

Cuando una página está bien estructurada:

- el mensaje se entiende mejor;
- la navegación es más fácil;
- el contenido se ve ordenado;

- La experiencia de la persona mejora;
- La participación se facilita.

En contextos comunitarios o ciudadanos, esto es muy importante. Una herramienta puede fracasar no porque el tema sea malo, sino porque la información está mal organizada.

CSS: forma visual y comprensión

Junto a HTML aparece CSS. CSS es el lenguaje que permite mejorar la apariencia visual de la página. Con CSS puedes trabajar:

- Colores;
- Tamaños;
- Espacios;
- Alineación;
- Bordes;
- Organización visual.

Pero aquí hay una idea clave: el diseño no es adorno vacío.
El diseño también comunica.

¿Por qué importa lo visual?

Porque una interfaz confusa puede alejar a la persona usuaria.

Si una página tiene mala organización visual, el contenido puede perderse aunque sea valioso.

CSS ayuda a:

- Resaltar lo importante;
- Separar secciones;
- Mejorar lectura;
- Hacer formularios más claros;
- Facilitar comprensión.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes haber entendido que:

la información no siempre merece confianza solo porque circula mucho;
las emociones fuertes exigen verificación;
las plataformas influyen en la opinión pública;
la terminal es una puerta al trabajo técnico organizado;
HTML estructura el contenido;
CSS ayuda a que ese contenido sea más claro visualmente.

Tip de la semana

Antes de compartir una información, respira, revisa y compara.
Y antes de escribir código, piensa primero qué quieres comunicar.

Retos frecuentes y cómo resolverlos

Pensar que una página web se trata solo de “verse bonita”.
No. Una página web debe primero ser clara, comprensible y útil.

Semana 3

Git, formularios, Node.js, servidores, puertos y rutas

El propósito de esta semana

En esta semana empiezas a entrar en el corazón técnico del proceso.

Ya tienes una base sobre ciudadanía digital, verificación, estructura web y primeros conceptos.

Ahora comenzarás a comprender cómo se registra el trabajo técnico, cómo se reciben datos en una interfaz y cómo una aplicación responde desde el backend.

Esta semana es fundamental porque aquí se conecta el aprendizaje organizativo con el aprendizaje técnico.

Git: ordenar el proceso de aprendizaje

Uno de los contenidos principales es Git.

Git es una herramienta de control de versiones.

Eso significa que permite guardar el historial de cambios de un proyecto.

¿Por qué Git es tan importante?

Cuando una persona empieza a trabajar archivos sin control, suele terminar con copias desordenadas:

- Trabajo final;
- Trabajo final bueno;
- Trabajo final ahora sí;
- Trabajo final definitivo;
- Trabajo final definitivo 2.



Ese desorden no solo hace perder tiempo. También hace perder trazabilidad. Git resuelve eso creando un historial real.

¿Qué significa trazabilidad?

Trazabilidad significa poder seguir el rastro de lo que hiciste.

Es saber:

- Qué cambiaste;
- Cuándo lo cambiaste;
- Por qué lo cambiaste;
- Cómo evolucionó tu trabajo.

Esto es importante porque en este curso no solo importa el resultado final. También importa el proceso.

Git como disciplina de trabajo

Git no es solo una herramienta técnica. También enseña hábitos de orden.

Te ayuda a:

- Registrar avances;
 - No perder cambios;
 - Volver atrás si algo falla;
 - Compartir un proyecto con más claridad;
 - Entender cómo progresa una práctica.
- Aprender Git es aprender a trabajar con más seriedad.

Conceptos básicos de Git para el estudiante

Repositorio

Es el lugar donde Git guarda el historial del proyecto.

Commit

Es un registro de cambios en un momento específico.

Historial

Es la secuencia de commits que muestra la evolución del proyecto.

Estado

Es la situación actual de los archivos: qué cambió, qué falta registrar, qué ya quedó guardado.

El valor pedagógico de Git

En este curso, Git ayuda a que el aprendizaje no sea invisible. Permite dejar evidencia. Eso es importante porque muchas veces una persona aprende mucho durante el proceso, aunque el resultado final todavía no sea perfecto.

Formularios: cuando una interfaz recibe información

Otro tema muy importante de esta semana son los formularios. Un formulario permite que una persona escriba y envíe información.

Puede servir para:

- Contacto;
- Registro;
- Inscripción;
- Reporte;
- Comentario;
- Solicitud;
- Participación.

¿Por qué los formularios son tan importantes?

Porque gran parte de la vida digital organizada se basa en formularios.

Una plataforma comunitaria puede usar formularios para:

- recibir reportes;
- conocer necesidades;
- recoger opiniones;
- registrar casos;
- canalizar solicitudes.

Por eso debes aprender a ver el formulario no solo como una caja para escribir, sino como un puente entre la persona y el sistema.

Qué hace que un formulario sea claro

Un buen formulario debe:

- Pedir solo lo necesario;
- Usar nombres de campos comprensibles;
- Estar ordenado;
- Indicar claramente qué se espera;
- Mostrar mensajes simples;
- Confirmar cuando una acción fue exitosa.

El principio de pedir menos

En tecnología responsable, pedir demasiados datos es un problema.

Puede cansar a la persona, generar confusión o exponer privacidad.

Por eso una regla importante es esta: pide solamente la información que realmente necesitas.

Node.js: JavaScript fuera del navegador

Esta semana introduces Node.js.

Node.js permite ejecutar JavaScript fuera del navegador.

Eso significa que JavaScript ya no solo se usa para la parte visible de la página, sino también para trabajar del lado del servidor.

¿Por qué esto es importante?

Porque ayuda a entender que una solución digital tiene dos capas:

- Una capa visible;
- Una capa interna.

La capa visible puede mostrar una interfaz.

La capa interna puede:

- Recibir solicitudes;
- Validar datos;
- Guardar información;
- Responder con resultados;
- Procesar lógica.

Node.js te ayuda a entrar en esa capa interna.

¿Para qué puede servir Node.js?

Node.js sirve para:

- Crear servidores;
- Responder solicitudes;
- Trabajar con rutas;
- Recibir datos;
- Organizar lógica;
- Construir APIs;
- Conectar con bases de datos.

El servidor: una idea esencial

Un servidor es un sistema que escucha solicitudes y responde.

Cuando una persona entra a una dirección web, ocurre una petición. El servidor responde con contenido o información.

Por eso, comprender el servidor es comprender que una aplicación no se limita a verse. También necesita responder.

Puertos: la entrada lógica del servicio

Cuando trabajas localmente, muchas veces usas direcciones como:

localhost:3000

Ese número final es el puerto.

Un puerto es una puerta lógica por donde escucha un servicio.

No necesitas profundizar demasiado al principio, pero sí entender que ayuda a identificar por dónde está corriendo la aplicación.

Rutas: organizar las respuestas del sistema

Una ruta es una dirección específica que define qué respuesta da el servidor.

Por ejemplo:

- /
- /saludo
- /reportes

Cada ruta puede servir para algo distinto.

¿Por qué las rutas son tan útiles?

Porque ayudan a convertir necesidades humanas en respuestas técnicas.

Por ejemplo:

- Una ruta puede mostrar información general;
- Otra puede devolver un saludo;
- Otra puede recibir un reporte;
- Otra puede enviar datos en formato JSON.

Las rutas organizan la lógica del backend.

GET y POST: dos tipos básicos de acción

GET: se usa normalmente para consultar información.

Por ejemplo:

- Ver mensajes;
- Consultar reportes;
- Cargar una lista;
- Leer contenido.

POST: se usa normalmente para enviar o registrar información.

Por ejemplo:

- Enviar un formulario;
- Guardar un mensaje;
- Registrar una solicitud.

¿Por qué es importante diferenciar GET y POST?

Porque cada acción digital tiene una intención.

Si una persona quiere consultar, piensas en GET.

Si quiere enviar información, piensas en POST.

Esta diferencia te ayudará a diseñar sistemas con más claridad.

JSON: datos organizados para comunicar sistemas

JSON es una forma ordenada de representar datos.

Se usa mucho para el intercambio de información entre frontend y backend.

Por ejemplo, una persona puede enviar:

- Nombre;
- Mensaje;
- Fecha.

Y el sistema puede recibir esos datos en una estructura organizada.

¿Por qué JSON es importante para el estudiante?

Porque te ayuda a entender cómo una acción del usuario se convierte en información útil para una aplicación.

Lo que debes llevarte de esta semana

Al terminar esta semana debes entender que:

Git organiza tu proceso;

los formularios conectan personas con sistemas;
Node.js abre la puerta al backend;
el servidor escucha y responde;
las rutas organizan funciones;
GET consulta y POST registra;
JSON ordena la información.

Tip de la semana

No aprendas Git, formularios o rutas como temas aislados. Intenta ver cómo se conectan.

Retos frecuentes y cómo resolverlos

Crear que backend es algo demasiado lejano o imposible.

No.

Backend empieza cuando entiendes que una acción del usuario necesita una respuesta organizada del sistema.

Semana 4

Comunicación digital para la paz, la democracia y el uso responsable del lenguaje en internet

El propósito de esta semana

En esta semana vas a comprender que comunicar en internet no es solamente publicar contenido. Comunicar también es decidir qué decir, cómo decirlo, con qué intención hacerlo y qué efectos puede generar en otras personas.

Muchas veces se piensa que la comunicación digital consiste solo en subir publicaciones, responder mensajes o compartir información. Pero en realidad es mucho más que eso. La comunicación digital influye en la convivencia, en la forma en que se entiende una situación, en la confianza entre las personas y en la calidad del debate público.

Por eso esta semana es fundamental dentro del curso. Aquí comenzarás a entender que una tecnología útil no solo debe funcionar bien. También debe comunicar con claridad, respeto y responsabilidad.

¿Qué es comunicar en digital?



Comunicar en digital significa usar medios tecnológicos para transmitir mensajes, ideas, orientaciones, opiniones o información a otras personas.

Esto ocurre en muchos espacios:

- Redes sociales;
- Grupos de mensajería;
- Páginas web;
- Foros;
- Plataformas educativas;
- Formularios;
- Aplicaciones de notificación;
- Canales comunitarios.

Pero la comunicación digital no es solo un cambio de formato. No es lo mismo decir algo en persona que decirlo en una publicación que puede circular, replicarse y permanecer en línea. Por eso el entorno digital exige más cuidado.

¿Por qué la comunicación digital tiene tanto impacto?

Porque internet amplifica.

Amplifica mensajes.

Amplifica errores.

Amplifica conflictos.

Amplifica malentendidos.

Pero también puede amplificar soluciones, convocatorias, aprendizajes y acuerdos.

Un mensaje mal escrito puede generar confusión.

Un mensaje agresivo puede detonar una discusión mayor.

Una información clara y bien presentada puede ayudar a una comunidad entera.

Eso significa que el lenguaje también es una herramienta tecnológica.

Comunicación para la paz: qué significa realmente

Hablar de paz en comunicación digital no significa evitar todas las diferencias ni convertir toda conversación en algo artificialmente amable. Significa aprender a comunicar sin aumentar innecesariamente el daño.

La paz en entornos digitales implica:

- No responder con violencia automática;
- No usar la humillación como forma de debate;
- No compartir rumores para destruir la reputación de alguien;
- No transformar una diferencia en una campaña de agresión;
- No usar el lenguaje para incendiar la conversación.

La paz digital también implica aprender a discutir con firmeza, pero sin deshumanizar al otro.

La diferencia entre debatir y agredir

En los entornos digitales muchas personas confunden debate con ataque.

Debatir significa:

- Exponer una idea;
- Argumentar;
- Escuchar otra posición;
- Responder con razones;
- Sostener una diferencia con claridad.

Agredir significa:

- Insultar;
- Ridiculizar;
- Descalificar;
- Humillar;
- Reducir a la otra persona a una etiqueta.

Esta diferencia es muy importante porque una plataforma comunitaria o ciudadana debe facilitar el intercambio respetuoso, no la destrucción del otro.

Comunicación democrática

La comunicación democrática es aquella que ayuda a que más personas comprendan, participen y se expresen con mayor claridad.

Una comunicación digital con enfoque democrático:

- Usa lenguaje comprensible;
- Presenta información suficiente;
- No busca manipular;
- Da espacio a la participación;
- Reconoce la diferencia;
- Cuida la precisión del mensaje;
- Evita la agresión como estrategia.

En otras palabras, una comunicación democrática no busca controlar a la audiencia. Busca informar, conectar y permitir participación más consciente.

El lenguaje claro como herramienta de inclusión

Un punto muy importante de esta semana es entender que comunicar bien no significa sonar complicado. Muchas veces ocurre lo contrario. Cuando una herramienta digital usa lenguaje demasiado técnico, demasiadas palabras confusas o instrucciones poco claras, excluye a muchas personas. Por eso el lenguaje claro es una forma de inclusión.

Lenguaje claro significa:

- Frases comprensibles;
- Instrucciones directas;
- Palabras conocidas;
- Mensajes concretos;
- Explicaciones ordenadas.

Esto importa mucho en páginas web, formularios, sistemas de reporte y plataformas comunitarias.

¿Por qué una mala comunicación puede dañar una herramienta útil?

Porque una herramienta puede estar bien construida técnicamente, pero fracasar si las personas no entienden qué hacer.

Por ejemplo:

- Si un formulario no explica qué pide;
- Si una página no indica el paso siguiente;
- Si un botón no tiene un nombre claro;
- Si el mensaje de confirmación es confuso;
- Si una instrucción parece ambigua.

Todo eso afecta la experiencia de la persona usuaria.

Por eso comunicar bien también es diseñar bien.

Mensajes que construyen confianza

En entornos comunitarios, una herramienta útil debe transmitir confianza.

Eso se logra cuando:

- La información está bien organizada;
- Las instrucciones son claras;
- El lenguaje es respetuoso;
- Los mensajes no exageran;
- El sistema responde con coherencia;
- La persona entiende qué pasó y qué sigue.

La confianza no se construye solo con tecnología. También se construye con forma, tono y claridad.

El valor de pensar en la audiencia

Otro aprendizaje importante de esta semana es que ningún mensaje existe en el vacío.

Siempre debes pensar:

- Quién lo va a leer;
- Qué nivel de conocimiento tiene;
- Qué necesita entender;
- Qué dudas puede tener;
- Qué emoción puede producir el mensaje;
- Qué acción se espera después.

Esta forma de pensar te servirá mucho más adelante cuando diseñes interfaces, formularios, instrucciones y plataformas completas.

Comunicación digital y estructura del contenido

Esta semana también conecta con lo técnico porque te obliga a pensar cómo se organiza la información en un entorno digital.

Un contenido bien comunicado normalmente tiene:

- Un propósito claro;
- Una estructura comprensible;
- Títulos o bloques diferenciados;
- Un mensaje principal reconocible;
- Una acción o cierre claro.

Eso ya te prepara para seguir mejorando en HTML, CSS y organización de interfaces.

La relación entre comunicación y tecnología social

Una tecnología social no es solamente un sistema que funciona. Es una herramienta que logra ayudar a personas reales en contextos reales.

Por eso la comunicación hace parte de su funcionamiento.

Si una plataforma organiza reportes, pero nadie entiende cómo reportar, la herramienta falla.

Si una página ofrece información, pero el mensaje es confuso, la herramienta falla.

Si una interfaz convoca participación, pero el lenguaje excluye o confunde, la herramienta falla.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes comprender que:

comunicar también es diseñar experiencia;

el lenguaje puede construir o destruir convivencia;

la paz digital no significa silencio, sino responsabilidad;

una comunicación democrática ayuda a participar mejor;

el lenguaje claro es parte de la inclusión.

Tip de la semana

Cuando escribas para una página, formulario o mensaje digital, imagina a una persona que nunca ha visto esa herramienta. Si esa persona entiende qué hacer, vas bien.

Retos frecuentes y cómo resolverlos

Pensar que la comunicación es solo “relleno” alrededor de la parte técnica.

No.

La comunicación es parte central del funcionamiento de una solución digital.

Semana 5

APIs, JSON y organización de la información para soluciones digitales útiles

El propósito de esta semana

Esta semana te ayuda a entender cómo circula la información entre diferentes partes de una solución digital.

Hasta ahora ya has visto que una aplicación tiene una parte visible y una parte interna. También viste que una persona puede llenar formularios, consultar información y enviar datos.

Ahora vas a comprender mejor cómo esos datos se organizan, cómo viajan y cómo pueden ser ofrecidos o recibidos por sistemas de forma estructurada.

Esta semana es importante porque te enseña a pensar la información no solo como contenido, sino como datos organizados con propósito.

Datos: de la intuición al orden

En la vida diaria tratamos con datos todo el tiempo.

Por ejemplo:

- Un nombre;
- Una fecha;
- Un mensaje;
- Una dirección;
- Un estado;



- Una categoría;
- Una observación.

Pero en una solución digital esos datos no pueden quedar dispersos o desordenados. Necesitan una estructura. Si no se organizan bien, el sistema se vuelve difícil de entender, difícil de mantener y difícil de usar.

JSON: una forma sencilla de organizar información

Uno de los conceptos centrales de esta semana es JSON.

JSON es un formato de texto que sirve para representar datos de forma ordenada. No necesitas memorizarlo de inmediato. Lo importante es entender para qué sirve. JSON permite que la información tenga una forma clara cuando viaja entre partes de un sistema.

Por ejemplo, si una persona llena un formulario con:

- Nombre;
- Mensaje;
- Fecha;

esa información puede organizarse en una estructura clara para que el backend la reciba, la procese y la responda.

¿Por qué JSON es tan importante?

Porque ayuda a convertir acciones humanas en datos útiles para el sistema.

La persona usuaria ve campos y botones.

El sistema ve información organizada.

JSON ayuda a construir ese puente.

Por eso este concepto es tan importante en aplicaciones modernas.

El valor de la estructura

Esta semana también te enseña una idea esencial: la información no debe circular de cualquier manera.

Una buena estructura ayuda a:

- Evitar confusión;
- Mantener orden;
- Facilitar consultas;
- Permitir respuestas claras;
- Mejorar la lógica interna;
- Conectar frontend y backend de manera más estable.

Cuando los datos tienen orden, el sistema puede crecer mejor.

¿Qué es una API?

Otro concepto central de esta semana es la API.

Una API es una forma organizada de permitir que un sistema ofrezca o reciba información.

No tienes que pensarla como algo abstracto o lejano. Puedes entenderla como una puerta de comunicación entre partes de una solución.

Por ejemplo:

- Una interfaz puede pedir datos a una API;
- Un formulario puede enviar un registro a una API;
- Un sistema puede consultar actualizaciones a través de una API.

¿Por qué las APIs son tan útiles?

Porque permiten que la información circule con estructura.

Sin una lógica organizada, cada conexión entre sistemas sería un desorden.

Las APIs ayudan a definir:

- Qué se puede pedir;
- Qué se puede enviar;
- Cómo debe enviarse;
- Qué respuesta devuelve el sistema.

Eso permite más claridad, más orden y más reutilización.

APIs y utilidad social

En este curso no se estudian APIs como un tema solo técnico. Se estudian como una forma de construir herramientas útiles.

Por ejemplo, una API podría servir para:

- Registrar reportes;
- Consultar información comunitaria;
- Mostrar actualizaciones;
- Organizar mensajes;
- Integrar un formulario con un sistema de seguimiento.

Lo importante es entender que las APIs ayudan a que una herramienta no se quede aislada.

De la interfaz a la lógica

Hasta ahora has visto que una persona puede interactuar con una página. Esta semana te ayuda a comprender que detrás de esa interacción hay un flujo.

Ese flujo puede verse así:

1. Una persona escribe datos;
2. La interfaz organiza la entrada;
3. Esos datos se envían;
4. El backend los recibe;
5. El sistema los procesa;
6. Se devuelve una respuesta.

JSON y API te ayudan a comprender mejor ese recorrido.

La diferencia entre mostrar y procesar

Una interfaz puede mostrar información, pero no siempre procesa la lógica completa.

Procesar puede incluir:

- Validar datos;
- Organizarlos;
- Decidir qué hacer con ellos;
- Almacenarlos;
- Responder con un resultado.

Por eso esta semana también fortalece la comprensión de la diferencia entre lo visible y lo interno.

Consultar y registrar información

Esta semana refuerza además el sentido de GET y POST.

Consultar

Consultar es pedir información.

Por ejemplo:

- Ver un listado;
- Leer una actualización;
- Consultar una noticia;
- Revisar un estado.

Registrar

Registrar es enviar información al sistema.

Por ejemplo:

- Enviar un mensaje;
- Crear un reporte;
- Guardar una solicitud;
- Registrar un dato.

Esta diferencia parece simple, pero es muy importante para pensar soluciones digitales con lógica.

La importancia de pedir solo lo necesario

A medida que empiezas a trabajar con datos, debes reforzar una regla muy importante: no toda información debe pedirse.

Una solución responsable debe preguntarse:

- ¿Realmente necesito este dato?
- ¿Para qué lo voy a usar?
- ¿Es sensible?
- ¿Podría generar riesgo si se expone?
- ¿Estoy complicando innecesariamente la experiencia?

Esta semana también ayuda a desarrollar ese criterio.

El diseño de la información

Así como una página debe tener diseño visual, la información también necesita diseño.

Diseñar información significa:

- Definir qué campos existen;
- Decidir qué dato va primero;
- Elegir nombres claros;
- Mantener coherencia;
- Facilitar lectura y uso.

Esto será clave más adelante cuando trabajes con bases de datos, autenticación y control de acceso.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes entender que:

los datos necesitan estructura;

JSON ayuda a organizar información;

las APIs permiten comunicación entre partes de un sistema;

consultar y registrar no son lo mismo;

la lógica de una solución digital depende de cómo organiza lo que recibe y lo que devuelve.

Tip de la semana

Cada vez que veas un formulario, pregúntate qué datos recoge, cómo podrían organizarse y qué respuesta debería devolver el sistema.

Retos frecuentes y cómo resolverlos

Pensar que los datos son solo “contenido” sin estructura.

No.

Los datos son parte de la arquitectura de una solución digital.

Semana 6

Comunidades digitales, participación ciudadana y herramientas para organizar necesidades colectivas

El propósito de esta semana

En esta semana el curso vuelve a enfocarse con más fuerza en la dimensión social de la tecnología.

Ya has aprendido que las plataformas no son neutras, que la comunicación digital tiene impacto y que la información necesita estructura.

Ahora vas a comprender mejor qué significa construir comunidad en espacios digitales y cómo la tecnología puede apoyar procesos de participación ciudadana. Esta semana es clave porque te ayuda a dejar de pensar en herramientas aisladas y comenzar a pensar en personas, vínculos, objetivos compartidos y procesos colectivos.

¿Qué es una comunidad digital?

Una comunidad digital es una red de personas que comparten:

- Necesidades;



Una comunidad digital no es simplemente un grupo de personas conectadas a internet. Tampoco es solamente un grupo de chat o una red social.

- Intereses;
- Temas de conversación;
- Objetivos;
- Mensajes;
- Reglas de interacción;
- Formas de participación.

Eso significa que una comunidad no nace solo porque exista una plataforma. Nace cuando hay sentido compartido.

El error de pensar que abrir un grupo ya crea comunidad

Este es uno de los errores más comunes.

A veces alguien crea:

- Un grupo;
- Una página;
- Un canal;
- Una plataforma;
- Un formulario;

y piensa que eso por sí solo ya construyó comunidad. No es así.

Una comunidad necesita:

- Propósito;
- Confianza;
- Continuidad;
- Reglas básicas;
- Canales claros;
- Lenguaje compartido;
- Escucha;
- Participación real.

La herramienta puede ayudar, pero no reemplaza el trabajo humano y organizativo.

¿Por qué la tecnología puede ayudar a una comunidad?

La tecnología puede ser muy útil cuando se usa con criterio.

Puede ayudar a:

- Difundir información;
- Organizar reportes;
- Convocar personas;
- Registrar necesidades;
- Dar seguimiento a procesos;
- Compartir orientaciones;
- Facilitar acceso a recursos;
- Visibilizar temas colectivos.

Pero para que eso ocurra, la herramienta debe responder a una necesidad real.

La importancia de comenzar por la necesidad

Cuando se piensa una solución digital para una comunidad, la primera pregunta no debe ser: “¿qué tecnología vamos a usar?”

La primera pregunta debería ser: “¿qué necesita esta comunidad?”

Por ejemplo:

- ¿Necesita difundir información clara?
- ¿Necesita recibir reportes?
- ¿Necesita coordinar acciones?
- ¿Necesita registrar casos?
- ¿Necesita hacer seguimiento?
- ¿Necesita reducir la confusión?

Solo después de comprender la necesidad tiene sentido pensar en formularios, páginas, canales o automatizaciones.

Participación ciudadana en entornos digitales

La participación ciudadana no ocurre solamente en reuniones presenciales o espacios institucionales. También puede fortalecerse mediante herramientas digitales.

Una persona participa digitalmente cuando:

- Se informa con criterio;
- Aporta ideas;
- Responde convocatorias;
- Registra situaciones;
- Da seguimiento a procesos;
- Consulta información pública;
- Propone mejoras;
- Se organiza con otras personas.

Por eso una solución digital puede ayudar mucho, siempre que facilite participación real y no solo apariencia de participación.

Participación real versus participación superficial

Una plataforma puede parecer muy activa sin que eso signifique participación profunda.

Por ejemplo, tener muchas reacciones o comentarios no siempre equivale a procesos comunitarios sólidos.

La participación real suele implicar:

- Comprensión del tema;
- Posibilidad de intervenir;
- Acceso a información clara;
- Seguimiento;
- Organización;
- Sentido de pertenencia.

Esta distinción es importante porque enseña al estudiante a no confundir visibilidad con participación verdadera.

Tecnología con enfoque comunitario

Pensar una tecnología con enfoque comunitario significa preguntarse:

- ¿Esto ayuda de verdad?
- ¿Las personas lo entienden?
- ¿Facilita acciones útiles?
- ¿Reduce barreras?
- ¿Ordena procesos?
- ¿Genera confianza?
- ¿Permite seguimiento?

No toda tecnología útil es la más compleja. Muchas veces la mejor solución es la más clara.

Identidad y lenguaje común

Las comunidades también se fortalecen cuando tienen identidad.

Esa identidad puede construirse mediante:

- Un propósito compartido;
- Mensajes consistentes;
- Lenguaje comprensible;
- Claridad sobre lo que se busca;
- Reglas de relación;
- Formas de nombrar problemas y soluciones.

Esto también se conecta con la comunicación digital responsable vista antes.

Herramientas que apoyan comunidad

En esta semana el estudiante debe empezar a reconocer que distintas herramientas cumplen funciones distintas.

Por ejemplo:

- Una página puede informar;
- Un formulario puede recoger necesidades;
- Un canal puede difundir avisos;
- Una tabla puede organizar seguimiento;
- Una automatización puede apoyar una notificación;
- Una base de datos puede conservar memoria del proceso.

La comunidad no depende de una sola pieza. Depende de cómo esas piezas ayudan a sostener el vínculo y la organización.

Pensar desde la experiencia de la persona usuaria

Una herramienta comunitaria debe ser pensada desde quien la usará.

Eso significa preguntarse:

- ¿La persona entenderá lo que debe hacer?
- ¿La información está clara?
- ¿El lenguaje es sencillo?
- ¿Hay demasiados pasos?
- ¿Se siente útil?
- ¿La respuesta es comprensible?

Esta lógica será muy importante más adelante, especialmente al hablar de inclusión y acceso comprensible.

Lo que debes llevarte de esta semana

Al terminar esta semana debes comprender que:
una comunidad digital es más que una plataforma;
la tecnología puede apoyar participación, pero no reemplaza el sentido colectivo;
una herramienta útil debe responder a necesidades reales;
la participación ciudadana requiere claridad, organización y seguimiento;
pensar en la comunidad significa pensar en las personas que usarán la solución.

Tip de la semana

Antes de diseñar cualquier herramienta, escribe primero cuál es la necesidad concreta que intentas resolver.

Retos frecuentes y cómo resolverlos

Pensar que una comunidad se construye solo con presencia digital.

No.

La comunidad se construye con propósito, confianza y procesos sostenidos. La tecnología solo debe ayudar a eso.

Semana 7

Redes de veeduría, acceso a la información y seguimiento organizado con apoyo tecnológico

El propósito de esta semana

Esta semana te ayuda a comprender cómo la tecnología puede apoyar procesos de observación, organización y seguimiento en asuntos de interés colectivo.

Aquí el curso se enfoca en una idea muy importante: participar no es solamente opinar. También implica observar, registrar, comprender y hacer seguimiento. Por eso estudiarás redes de veeduría, acceso a la información y el valor de la trazabilidad en procesos comunitarios y ciudadanos.

¿Qué es una red de veeduría?

Una red de veeduría es un conjunto de personas o actores que observan, registran y hacen seguimiento a temas de interés colectivo.

Su propósito no es generar ruido, sino ayudar a que exista mayor claridad y control social sobre procesos que afectan a una comunidad.

Una red de veeduría puede interesarse por temas como:

- Acceso a servicios;
- Seguimiento de compromisos;
- Situaciones comunitarias;
- Reportes ciudadanos;



- Alertas;
- Gestión local;
- Necesidades colectivas.

La relación entre veeduría y tecnología

La tecnología puede ser muy útil para apoyar una red de veeduría porque ayuda a organizar información.

Por ejemplo, puede servir para:

- Recibir reportes;
- Registrar casos;
- Clasificar situaciones;
- Hacer seguimiento a estados;
- Compartir actualizaciones;
- Generar trazabilidad;
- Visibilizar patrones.

Esto conecta muy bien con todo lo que has aprendido antes sobre formularios, datos, rutas, API y organización de la información.

¿Qué significa hacer seguimiento?

Hacer seguimiento significa no dejar que una situación desaparezca después del primer registro.

Por ejemplo, si una persona reporta algo importante, el seguimiento permite preguntarse:

- ¿Qué pasó después?
- ¿Se revisó el caso?
- ¿Hubo respuesta?
- ¿Se actualizó el estado?
- ¿Sigue pendiente?
- ¿Quién debe ver esto?

Sin seguimiento, la información se acumula pero no se convierte en proceso útil.

El valor de la trazabilidad en contextos comunitarios

Esta semana también refuerza la idea de trazabilidad. Trazabilidad significa poder seguir el recorrido de una información o de una acción.

Eso ayuda a saber:

- Cuándo se registró algo;
- Qué se hizo después;
- Quién lo revisó;
- Si hubo cambios;
- Si sigue pendiente;
- Qué decisiones se tomaron.

En contextos ciudadanos, esto es muy valioso porque ayuda a evitar pérdida de

información, improvisación y desorden.

Acceso a la información: base para participar mejor

Una comunidad participa mejor cuando entiende qué está ocurriendo. Por eso el acceso a la información es tan importante.

No basta con que exista información. Debe estar:

- Disponible;
- Ordenada;
- Clara;
- Actualizada;
- Comprensible.

Si la información está desorganizada o resulta confusa, la participación se debilita.

Información clara versus información acumulada

A veces se cree que tener mucha información es suficiente. Pero una gran cantidad de datos mal organizada puede ser tan inútil como no tener nada. Una buena solución tecnológica no solo muestra datos. También ayuda a entenderlos.

Eso puede implicar:

- Clasificar;
- Resumir;
- Estructurar;
- Nombrar bien;
- Presentar estados;
- Indicar pasos siguientes.

El estudiante como futuro organizador de información

Esta semana también es importante porque te enseña a asumir un rol más activo. No eres solo alguien que consulta herramientas. También puedes aprender a diseñar herramientas que permitan ordenar mejor la información colectiva.

Eso requiere pensar preguntas como:

- ¿Qué dato se registra?
- ¿Cómo se clasifica?
- ¿Quién lo consulta?
- ¿Qué seguimiento debe tener?
- ¿Cómo se evita el caos?

Tecnología para comprender, no solo para almacenar

Una herramienta comunitaria útil no se limita a guardar registros.

También debe permitir:

- Comprender situaciones;
- Identificar prioridades;
- Reconocer patrones;

- Hacer seguimiento;
- Orientar decisiones.

Esto es lo que transforma una acumulación de información en una plataforma útil.

Formularios, estados y seguimiento

Aquí se consolidan varias ideas anteriores. Un formulario puede servir para registrar un hecho.

Pero después de ese registro, también puede ser necesario:

- Asignar una categoría;
- Indicar un estado;
- Registrar observaciones;
- Actualizar seguimiento;
- Consultar el historial.

Esta lógica prepara muy bien el paso a temas posteriores como bases de datos, CRUD y control de acceso.

El vínculo entre comunidad, información y confianza

Cuando una comunidad ve que la información se registra con orden y puede revisarse con claridad, aumenta la confianza.

La confianza crece cuando las personas sienten que:

- Lo que reportan no se pierde;
- Hay seguimiento;
- La información es comprensible;
- El sistema tiene coherencia;
- Los pasos son visibles.

Por eso una herramienta tecnológica también puede fortalecer confianza institucional o comunitaria si está bien pensada.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes comprender que:
participar también implica observar y hacer seguimiento;
las redes de veeduría necesitan orden informativo;
la trazabilidad ayuda a no perder el sentido del proceso;
el acceso a la información fortalece participación;
una herramienta útil no solo registra, también ayuda a comprender.

Tip de la semana

Cuando pienses en una plataforma comunitaria, no pienses solo en cómo entra la información. Piensa también en cómo se seguirá, se actualizará y se consultará después.

Retos frecuentes y cómo resolverlos

Crear que registrar información ya es suficiente.

No.

Registrar es apenas el comienzo. Lo que da valor real es el seguimiento y la capacidad de comprender lo registrado.

Semana 8

Software de código abierto, sistemas operativos y comprensión del entorno donde vive una aplicación

El propósito de esta semana

En esta semana vas a mirar una parte del mundo tecnológico que muchas veces pasa desapercibida para quien apenas empieza: el entorno donde realmente funciona una aplicación.

Hasta ahora has venido aprendiendo sobre plataformas, información, formularios, rutas, comunidad y organización de datos. Pero una solución digital no existe en el aire. Necesita una base que la sostenga.

Por eso esta semana se concentra en dos temas grandes:

- El software de código abierto;
- Los sistemas operativos.

Estos temas son importantes porque te ayudan a comprender que la tecnología no es magia. Funciona sobre estructuras concretas, con herramientas que tienen historia, lógica y condiciones de uso.

¿Qué significa software de código abierto?

El software de código abierto es aquel cuyo funcionamiento puede revisarse, estudiarse, modificarse o reutilizarse según las condiciones de su licencia.



Esto quiere decir que no siempre estás frente a una “caja cerrada”. En muchos casos, existen herramientas construidas con una lógica de colaboración, transparencia y mejora continua.

¿Por qué importa entender esto?

Porque te ayuda a ver la tecnología de otra manera. No todo está hecho para que simplemente lo consumas. Muchas herramientas tecnológicas también pueden analizarse, entenderse y adaptarse. Eso cambia tu posición como estudiante.

Ya no eres solamente alguien que usa herramientas. Empiezas a comprender que la tecnología también puede ser examinada, mejorada y construida desde procesos colaborativos.

Código abierto y aprendizaje

El código abierto tiene un valor pedagógico muy grande.

Ayuda a:

- Explorar cómo están hechas las herramientas;
- Aprender observando estructuras reales;
- Comprender que el desarrollo tecnológico también puede ser colectivo;
- Reconocer que el conocimiento puede circular de manera más abierta.

Para quien está comenzando, esta idea es muy valiosa porque rompe la sensación de que todo lo técnico es inaccesible o reservado para unas pocas personas.

¿Qué es un sistema operativo?

Un sistema operativo es la base que permite que un computador o un dispositivo funcione de manera organizada.

Es el entorno que administra:

- Programas;
- Archivos;
- Procesos;
- Memoria;
- Recursos del sistema;
- Comunicación con el hardware.

En palabras más simples: el sistema operativo es el “gran organizador” del equipo. Gracias a él puedes abrir aplicaciones, guardar archivos, moverte entre carpetas, ejecutar procesos y trabajar con herramientas técnicas.

Ejemplos de sistemas operativos

Algunos sistemas operativos conocidos son:

- Windows;
- Linux;
- MacOS;
- Android;
- iOS.

Aunque no todos se usan igual en desarrollo, todos ayudan a entender la idea general: una aplicación necesita un entorno para poder ejecutarse.

¿Por qué este tema es importante en un curso como este?

Porque muchas personas aprenden a usar aplicaciones sin preguntarse dónde corren o qué las sostiene. Pero si quieres comprender mejor la tecnología, necesitas mirar también el suelo donde esa tecnología se apoya.

Por ejemplo:

- Git corre sobre un sistema;
- Node.js se ejecuta en un sistema;
- Los archivos del proyecto viven en un sistema;
- La terminal se comunica con el sistema;
- Un servidor funciona dentro de un sistema.

Entender esto te da una mirada más completa.

La aplicación no vive sola

Una de las ideas más importantes de esta semana es esta: una aplicación no vive sola.

Necesita:

- Archivos;
- Carpetas;
- Procesos;
- Permisos;
- Configuración;
- Entorno de ejecución.

Cuando comienzas a entender esto, dejas de ver la aplicación como algo aislado y empiezas a verla como parte de un ecosistema técnico.

Archivos, carpetas y orden técnico

El sistema operativo también te ayuda a organizar el trabajo. Un proyecto no puede crecer bien si sus archivos están desordenados.

Por eso esta semana también refuerza una idea de disciplina técnica:

- Poner nombres claros;
- Separar carpetas;
- Organizar archivos;
- No mezclar todo en un solo lugar;
- Entender qué guarda cada parte del proyecto.

Esta organización parece simple, pero tiene mucho valor en la práctica real.

Procesos: lo que está corriendo

Otro concepto útil de esta semana es el de proceso. Un proceso es una tarea que el sistema está ejecutando.

Por ejemplo:

- Un editor abierto;
- Un servidor corriendo;
- Una terminal ejecutando un comando;
- Una aplicación funcionando.

Comprender que una herramienta “está corriendo” como proceso te ayuda a entender mejor por qué a veces un sistema se detiene, por qué algo consume recursos o por qué debes iniciar o cerrar ciertos servicios.

El sistema operativo como base de comprensión

Esta semana no busca que te conviertas en administrador de sistemas. Busca algo más importante para esta etapa: que comprendas que toda herramienta tecnológica se apoya en una base concreta.

Esto te ayuda a:

- Perder miedo al entorno técnico;
- Entender mejor la terminal;
- Comprender por qué importa la instalación correcta;
- Reconocer que una aplicación necesita contexto;
- Trabajar con mayor conciencia del ambiente donde desarrollas.

La relación entre código abierto y autonomía

Cuando estudias código abierto y sistemas operativos también comienzas a desarrollar autonomía. Autonomía significa no depender siempre de explicaciones mágicas o respuestas memorizadas.

Significa poder preguntarte:

- ¿Dónde está este archivo?
- ¿Qué está ejecutando este programa?
- ¿Por qué esta herramienta funciona aquí?
- ¿Qué necesita para correr?
- ¿Qué parte depende del sistema?

Estas preguntas son muy valiosas para cualquier estudiante que quiera crecer técnicamente.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes comprender que:

las herramientas tecnológicas corren sobre una base concreta;
el software de código abierto muestra que la tecnología también puede construirse colaborativamente;
el sistema operativo organiza archivos, procesos y recursos;

una aplicación necesita entorno, no solo código;
el orden técnico comienza por comprender dónde y cómo vive el proyecto.

Tip de la semana

Cuando uses una herramienta, no pienses solo en “qué hace”. Pregúntate también “dónde corre”, “qué necesita” y “cómo se organiza”.

Retos frecuentes y cómo resolverlos

Pensar que el sistema operativo es solo “la pantalla principal” del computador.
No.

El sistema operativo es una parte esencial del funcionamiento técnico de cualquier proyecto.

Semana 9

Seguridad digital, protección básica, variables de entorno y cuidado del proyecto

El propósito de esta semana

En esta semana vas a estudiar un tema decisivo: la seguridad digital. La seguridad muchas veces se imagina como algo lejano, complejo o reservado para especialistas.

Sin embargo, en el aprendizaje inicial ya existen prácticas de seguridad que debes empezar a incorporar.

Esta semana busca que entiendas algo muy importante: una solución digital útil puede convertirse en un problema si no cuida la información, los accesos y los hábitos básicos de trabajo.

Por eso aquí no solo verás ideas técnicas. También verás hábitos responsables.

¿Qué es la seguridad digital?

La seguridad digital es el conjunto de prácticas que ayudan a reducir riesgos sobre:

- Información;
- Accesos;
- Cuentas;
- Archivos;
- Funcionamiento del sistema;
- Integridad del proyecto.



No significa vivir con miedo. Significa actuar con cuidado.

¿Por qué la seguridad importa desde el inicio?

Porque muchos errores de seguridad no aparecen cuando el sistema ya es muy grande. Aparecen desde los primeros pasos.

Por ejemplo:

- Compartir contraseñas;
- Dejar archivos sensibles visibles;
- Subir claves a un repositorio;
- Pedir datos innecesarios;
- Confiar sin revisar;
- Exponer más información de la que se necesita.

Por eso, esta semana te ayuda a construir hábitos sanos desde temprano.

Riesgos básicos que debes reconocer

Compartir credenciales

Uno de los errores más graves es compartir contraseñas o accesos sin control. Una contraseña no es un detalle menor. Es una llave.

Publicar secretos en el proyecto

- A veces una persona deja en el código:
- Claves;
- Tokens;
- Correos sensibles;
- Contraseñas;
- Configuraciones privadas.

Eso puede generar riesgos serios, especialmente si el proyecto se publica.

Pedir datos que no hacen falta

Cada vez que una plataforma pide información, debe existir una razón clara.

Pedir de más puede generar:

- Incomodidad;
- Desconfianza;
- Sobrecarga;
- Riesgo de exposición.

Abrir rutas o accesos sin criterio

No toda información debe estar disponible para cualquiera. Una solución sería empezar a pensar qué se puede ver y qué no.

Seguridad como hábito, no solo como tema técnico

Una idea muy importante de esta semana es que la seguridad no depende solo de herramientas sofisticadas. Muchas veces depende de hábitos.

Por ejemplo:

- Revisar antes de publicar;
- No copiar y pegar credenciales;
- No dejar información sensible en archivos abiertos;
- Entender qué información debe protegerse;
- Trabajar con orden;
- Evitar improvisación.

Eso significa que la seguridad también es cultura de trabajo.

Variables de entorno: con figuración fuera del código

Otro contenido muy importante de esta semana son las variables de entorno.

Una variable de entorno es un valor de configuración que se guarda por fuera del código.

¿Para qué sirve esto?

Sirve para evitar que información sensible quede escrita directamente en los archivos del proyecto. Por ejemplo, ciertas configuraciones no deberían quedar expuestas dentro del código visible.

Las variables de entorno ayudan a separar:

- La lógica del proyecto;
- La configuración sensible;
- Los datos según el entorno.

¿Por qué este concepto importa para el estudiante?

Porque te enseña desde temprano una práctica sana: no mezclar todo en un mismo lugar.

Esto mejora:

- El orden;
- La seguridad;
- La claridad del proyecto;
- La posibilidad de cambiar configuraciones sin alterar toda la lógica.

Datos sensibles y datos no sensibles

Esta semana también te ayuda a distinguir mejor entre datos sensibles y no sensibles. No toda información tiene el mismo nivel de riesgo.

Por ejemplo, pueden existir datos que son públicos o informativos, y otros que requieren mucho más cuidado. Aprender a distinguir esto es clave para cualquier solución responsable.

Minimización de datos

Una regla importante de esta semana es la minimización de datos. Eso significa pedir, guardar o mostrar solo lo necesario. Esta idea tiene mucho valor porque protege mejor a las personas y reduce riesgos.

Antes de pedir un dato, siempre conviene preguntarse:

- ¿Lo necesito de verdad?
- ¿Qué haré con él?
- ¿Qué pasa si se expone?
- ¿Podría resolver esto con menos información?

Revisar antes de publicar

En esta etapa del curso, revisar antes de publicar debe convertirse en un hábito.

Eso implica mirar:

- Qué archivos estás subiendo;
- Qué información aparece;
- Si hay claves visibles;
- Si el contenido es el correcto;
- Si lo que publicas tiene sentido y está limpio.

Este hábito sirve tanto para seguridad como para calidad.

Seguridad y confianza comunitaria

Si una herramienta comunitaria no cuida la información, la confianza se rompe. Por eso la seguridad no es solamente un tema técnico. También es un tema de credibilidad y responsabilidad.

Una comunidad confía más cuando percibe que la información:

- No se expone sin razón;
- Se maneja con cuidado;
- Está organizada;
- Se trata con seriedad.

La seguridad en proyectos sencillos también importa

A veces una persona piensa: “como esto es pequeño, no importa tanto”.

Ese pensamiento es peligroso. La seguridad no empieza cuando el proyecto ya es enorme. Empieza cuando construyes el hábito de trabajar bien.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes comprender que:

la seguridad digital comienza con hábitos básicos;
no toda información debe quedar visible en el proyecto;
las variables de entorno ayudan a separar configuración sensible;
pedir menos datos suele ser una mejor práctica;
la seguridad también protege la confianza en una herramienta comunitaria.

Tip de la semana

Cada vez que vayas a publicar algo, revisa si estás exponiendo información que no debería aparecer.

Retos frecuentes y cómo resolverlos

Pensar que la seguridad es un tema para “después”.

No.

La seguridad empieza desde el primer archivo, el primer formulario y el primer repositorio.

Semana 10

Metodologías ágiles, organización del trabajo, backlog e historias de usuario

El propósito de esta semana

En esta semana el curso se concentra en una habilidad que muchas veces se subestima: aprender a organizar el trabajo. Hasta aquí ya has visto conceptos sociales, herramientas técnicas, lógica de backend, estructuras de información, seguridad y comunidad. Ahora necesitas aprender a ordenar todo eso de forma más práctica.

Por eso esta semana introduce metodologías ágiles y conceptos que ayudan a dividir el trabajo en partes manejables. La idea central es muy valiosa: no intentar resolver todo de una vez.

¿Qué significa trabajar de manera ágil?

Trabajar de forma ágil significa avanzar por partes, revisar constantemente y mejorar sobre la marcha. No se trata de hacer las cosas con afán. Se trata de construir con ciclos cortos, objetivos claros y capacidad de ajuste.



Esto es muy útil para estudiantes porque evita el caos de querer hacer todo al mismo tiempo.

¿Por qué esto le sirve a una persona que apenas empieza?

Porque cuando alguien está comenzando, suele sentir que el proyecto completo es demasiado grande. Las metodologías ágiles enseñan a reducir esa ansiedad.

Ayudan a pensar así:

- ¿Qué puedo resolver primero?
- ¿Qué es lo más importante?
- ¿Qué puede esperar?
- ¿Cómo convierto una idea grande en pasos pequeños?

Esa forma de pensar te servirá mucho en cualquier proyecto.

Backlog: lista priorizada de trabajo

Uno de los conceptos más importantes de esta semana es el backlog.

El backlog es una lista organizada y priorizada de tareas o necesidades.

No es una lista caótica de pendientes. Debe tener orden.

¿Qué tipo de cosas puede incluir un backlog?

Por ejemplo:

- Crear una página de inicio;
- Hacer un formulario;
- Registrar un reporte;
- Mostrar una con firmación;
- Proteger una ruta;
- Mejorar el lenguaje de una interfaz;
- Organizar una base de datos;
- Documentar una práctica.

El backlog ayuda a ver el proyecto como piezas manejables.

Priorizar: no todo vale lo mismo al mismo tiempo

Otra idea clave es la priorización. Priorizar significa decidir qué tarea debe resolverse primero. Eso importa porque no todo tiene la misma urgencia ni el mismo impacto.

Por ejemplo, puede ser más importante primero lograr que el formulario funcione que dedicar mucho tiempo a detalles visuales menores. Aprender a priorizar ayuda a construir con más inteligencia.

Historia de usuario: necesidad expresada desde quien usa la herramienta

Otro concepto esencial de esta semana es la historia de usuario. Una historia de usuario es una forma breve de expresar una necesidad desde la perspectiva de quien usará la solución.

Suele seguir una estructura simple:

Como [tipo de usuario], quiero [acción], para [beneficio].

¿Por qué esto es tan valioso?

Porque obliga a pensar la herramienta desde la experiencia de la persona usuaria. No se trata solo de lo que el desarrollador quiere hacer. Se trata de lo que la persona realmente necesita.

Ejemplos sencillos

- Como ciudadano quiero registrar un reporte para dejar evidencia de una situación.
- Como coordinador quiero consultar los reportes para hacer seguimiento.
- Como persona usuaria quiero recibir con firmación después de enviar un formulario para saber que la acción sí fue registrada.

Estas historias ayudan a conectar tecnología con propósito real.

Iteración: trabajar por bloques cortos

Otra idea útil es la iteración. Una iteración es un bloque corto de trabajo en el que te concentras en un conjunto limitado de tareas. En vez de intentar resolver un sistema completo de una vez, avanzas en una parte concreta.

Por ejemplo, una iteración inicial podría centrarse solo en:

- Crear la interfaz;
- Construir el formulario;
- Mostrar un mensaje de con firmación.

Esto hace el trabajo más manejable y permite revisar mejor.

Retroalimentación y mejora continua

Trabajar por ciclos también significa revisar y mejorar. No se espera que todo salga perfecto al primer intento.

La retroalimentación sirve para:

- Identificar errores;
- Simplificar soluciones;
- Mejorar claridad;
- Corregir diseño;
- Ajustar la lógica.

Esto es especialmente valioso en contextos comunitarios, donde las necesidades reales pueden cambiar o aclararse con el tiempo.

Organización del trabajo y confianza personal

Aprender a organizar el trabajo también mejora la confianza del estudiante.

Cuando el proyecto parece enorme, aparece frustración. Cuando el proyecto se convierte en pequeñas tareas claras, el camino se vuelve más manejable.

Por eso esta semana también tiene valor emocional y pedagógico. Te enseña a no ahogarte en la complejidad.

Tecnología y escucha de necesidades reales

En proyectos con sentido comunitario, la organización del trabajo no puede ignorar a las personas usuarias. Por eso las historias de usuario y la priorización tienen tanto sentido.

Ayudan a construir herramientas que respondan a necesidades reales, no solo a ideas técnicas interesantes pero desconectadas.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes comprender que:
trabajar bien también implica organizar bien;
un backlog ayuda a ordenar tareas;
priorizar es decidir qué aporta más valor primero;
las historias de usuario conectan tecnología con necesidades reales;
las iteraciones permiten avanzar sin intentar resolver todo de una sola vez.

Tip de la semana

Si una tarea te parece demasiado grande, divídela hasta que se convierta en un paso manejable.

Retos frecuentes y cómo resolverlos

Pensar que planear es perder tiempo.

No.

Planear bien ahorra errores, desorden y frustración más adelante.

Semana 11

Automatización con n8n, uso de Telegram y apoyo tecnológico para tareas repetitivas

El propósito de esta semana

En esta semana vas a estudiar una idea muy práctica: no todo en tecnología debe hacerse siempre de forma manual. Existen tareas repetitivas que pueden organizarse mejor con automatización.

Aquí aparece n8n como herramienta para pensar flujos simples y también Telegram como posible canal de apoyo para notificaciones y coordinación.

Esta semana es importante porque te enseña a usar la tecnología no solo para crear interfaces o servidores, sino también para ahorrar tiempo y ordenar procesos.

¿Qué es automatizar?

Automatizar significa hacer que una tarea repetitiva pueda ejecutarse con menos intervención manual, siguiendo reglas definidas. No significa perder control. Significa ordenar mejor lo repetitivo.

Ejemplos de tareas repetitivas

- Enviar un aviso cuando entra un nuevo reporte;
- Mover información de un punto a otro;
- Notificar un cambio de estado;



- Registrar una acción automática;
- Organizar una alerta simple.

Estas tareas, cuando se repiten muchas veces, consumen tiempo y aumentan el riesgo de olvido o desorden.

n8n como herramienta de automatización

n8n es una herramienta que permite construir flujos automatizados. No necesitas entrar en máxima profundidad técnica al comienzo. Lo importante es entender su lógica.

Un flujo de automatización tiene normalmente:

- Un evento que activa;
- Una serie de pasos;
- Una acción final.

Pensar una automatización sencilla

Por ejemplo, una lógica simple podría ser:

1. Entra un nuevo reporte;
2. El sistema detecta el evento;
3. Se genera un mensaje corto;
4. Se envía una notificación;
5. Se registra que la alerta salió.

Este tipo de flujo ayuda a comprender cómo automatización y seguimiento pueden conectarse.

Automatizar con criterio

Una lección clave de esta semana es que no todo debe automatizarse. Automatizar no es un objetivo en sí mismo.

Debe automatizarse aquello que:

- Se repite mucho;
- Tiene reglas claras;
- No implica decisiones delicadas;
- No expone información sensible sin controles.

¿Qué debe automatizarse con cuidado?

Hay procesos que requieren mucha prudencia.

Por ejemplo, no es buena idea automatizar sin control temas que involucren:

- Datos personales sensibles;
- Decisiones críticas;
- Accesos delicados;
- Acciones que deberían tener validación humana;
- Información privada mal protegida.

Esta semana te enseña que automatizar también exige criterio ético y técnico.

Telegram como canal de coordinación

Telegram aparece aquí como un posible canal de apoyo.

Puede servir para:

- Enviar avisos;
- Compartir notificaciones generales;
- Apoyar seguimiento;
- Coordinar acciones sencillas;
- Recibir alertas de procesos.

No se presenta como la solución total, sino como un canal posible dentro de un ecosistema de trabajo.

La importancia de pensar el canal

No toda información debe circular por mensajería. Esta semana también ayuda a desarrollar criterio sobre los canales.

Preguntas importantes serían:

- ¿Qué tipo de información se va a enviar?
- ¿Es sensible o no?
- ¿Quién la recibirá?
- ¿Este canal es apropiado?
- ¿Qué pasa si el mensaje se reenvía?

Pensar esto es parte del uso responsable de herramientas.

Automatización y coordinación comunitaria

La automatización puede ser útil en contextos comunitarios si se usa de forma prudente.

Por ejemplo, puede ayudar a:

- Avisar sobre nuevos registros;
- Compartir recordatorios;
- Notificar actualizaciones no sensibles;
- Mejorar tiempos de respuesta;
- Evitar que ciertos pasos dependan completamente de la memoria humana.

Eso mejora orden y continuidad.

El riesgo de automatizar sin comprender

Uno de los errores más comunes es querer automatizar sin haber entendido bien el proceso manual. Eso suele generar caos.

Antes de automatizar, debes entender:

- Qué ocurre;
- Qué paso sigue a cuál;

- Qué datos intervienen;
- Qué parte sí puede hacerse sola;
- Qué parte necesita revisión humana.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes comprender que:

automatizar sirve para ordenar tareas repetitivas;

no ayuda a pensar flujos;

Telegram puede apoyar coordinación y avisos;

no toda tarea debe automatizarse;

una automatización útil ahorra tiempo sin poner en riesgo la información.

Tip de la semana

Antes de automatizar algo, dibuja primero el proceso manual paso a paso. Si no entiendes ese proceso, la automatización saldrá mal.

Retos frecuentes y cómo resolverlos

Pensar que automatizar siempre significa mejorar.

No.

Automatizar sin criterio puede amplificar errores o exponer información innecesariamente.

Semana 12

Bases de datos, CRUD, control de acceso, inclusión digital y proyecto integrador final

El propósito de esta semana

Esta última semana reúne muchas de las piezas que has venido trabajando a lo largo del curso. Aquí vas a comprender mejor cómo una solución digital guarda memoria, cómo organiza operaciones sobre datos, cómo controla quién entra y qué puede hacer cada persona, y por qué una herramienta solo tiene verdadero valor cuando es comprensible y útil para las personas. También es la semana donde todo esto se conecta en el proyecto integrador final.

Persistencia: que la información no desaparezca

Uno de los primeros temas de esta semana es la persistencia. Persistencia significa que la información pueda guardarse y mantenerse aunque el sistema se detenga o se cierre. Esto es importante porque una solución real necesita memoria.

Por ejemplo, una plataforma puede necesitar recordar:

- Usuarios;
- Reportes;
- Mensajes;
- Estados;
- Observaciones;



- Seguimientos.

Si la información desapareciera cada vez que se cierra el sistema, la herramienta perdería gran parte de su utilidad.

Bases de datos: memoria organizada del sistema

Una base de datos es un sistema organizado para guardar y consultar información.

No se trata simplemente de “guardar por guardar”. Se trata de conservar datos con orden, de manera que luego puedan:

- Consultarse;
- Actualizarse;
- Clasificarse;
- Relacionarse;
- Recuperarse.

Esta semana ayuda a que entiendas la base de datos como la memoria organizada de una plataforma.

¿Por qué este tema es importante?

Porque muchas herramientas comunitarias o ciudadanas necesitan más que una pantalla bonita. Necesitan recordar. Si una comunidad registra algo importante, ese dato no debería desaparecer.

Por eso el curso introduce este tema como una parte clave de cualquier solución con sentido de continuidad.

CRUD: cuatro acciones básicas sobre la información

Otro concepto central de esta semana es CRUD.

CRUD reúne cuatro operaciones muy comunes en los sistemas:

- Crear;
- Leer;
- Actualizar;
- Eliminar.

¿Por qué esto es útil para el estudiante?

Porque ayuda a ver que detrás de muchas interfaces hay acciones lógicas repetidas.

Por ejemplo, con reportes comunitarios podrías tener:

- Crear un reporte;
- Leer una lista de reportes;
- Actualizar el estado de un reporte;
- Eliminar o desactivar un registro.

Comprender CRUD ayuda a pensar sistemas con más claridad.

Autenticación: verificar quién eres

La autenticación es el proceso mediante el cual el sistema verifica la identidad de una persona. En otras palabras, responde a la pregunta: ¿quién está entrando?

Este concepto es importante porque no toda plataforma es totalmente abierta ni todas las acciones deben estar disponibles sin validación.

Autorización: definir qué puedes hacer

La autorización responde a otra pregunta: una vez que ya sabes quién eres, ¿qué puedes hacer?

Esto es distinto de autenticación. Puedes estar autenticado, pero no necesariamente autorizado para todo.

Ejemplo simple de roles

- Una persona usuaria puede registrar un reporte;
- Una persona coordinadora puede revisar y actualizar estados;
- Una persona administradora puede gestionar configuraciones o accesos.

Esta diferencia ayuda a proteger mejor la información y a mantener orden operativo.

¿Por qué el control de acceso importa en contextos ciudadanos?

Porque una plataforma sería necesaria definir límites y responsabilidades.

No toda persona debe ver lo mismo.

No toda persona debe modificar lo mismo.

No toda persona debe acceder a información sensible.

Esta semana te ayuda a comprender que el orden también es una forma de cuidado.

Inclusión digital: una herramienta debe poder ser entendida

Otro tema fundamental de esta semana es la inclusión digital. La inclusión digital no significa solo que exista una plataforma en internet. Significa que la plataforma sea comprensible y usable para más personas.

Una herramienta puede fallar por varias razones:

- Lenguaje demasiado técnico;
- Demasiados pasos;
- Formularios enredados;
- Estructura visual confusa;
- Instrucciones débiles;
- Navegación poco clara.

Por eso una solución útil debe pensar en personas reales, no solo en lógica técnica.

Tecnología clara versus tecnología complicada

Un error común es creer que una herramienta “más avanzada” es la que tiene más complejidad. No siempre.

En muchos casos, una herramienta mejor es aquella que:

- Se entiende rápido;
- Guía bien a la persona;
- Pide lo necesario;
- Responde con claridad;
- No genera confusión;
- Facilita la acción útil.

Eso también es calidad tecnológica.

Proyecto integrador final: reunir todo con sentido

El proyecto integrador final no es una simple suma de temas.

Es el momento en que debes demostrar que entendiste la relación entre:

- Tecnología;
- Información;
- Comunidad;
- Seguridad;
- Organización;
- Comunicación;
- Utilidad social.

¿Qué debe mostrar el proyecto final?

Debe mostrar que puedes:

- Organizar información de forma clara;
- Entender la diferencia entre interfaz y lógica interna;
- Construir una solución básica útil;
- Aplicar criterios de seguridad y cuidado;
- Pensar en personas usuarias reales;
- Dar sentido social a la herramienta.

El valor del proyecto no está en hacerlo “complicado”

Es muy importante que el estudiante entienda esto. El objetivo no es hacer algo innecesariamente grande o complejo.

El objetivo es hacer algo:

- Claro;
- Ordenado;
- Útil;
- Comprensible;
- Coherente con lo aprendido.

A veces un proyecto pequeño, pero bien pensado, demuestra mucho más aprendizaje que uno grande pero desordenado.

Cierre conceptual de todo el proceso

Esta última semana también te invita a mirar hacia atrás. A lo largo del curso has visto que aprender tecnología no es solo aprender comandos, etiquetas o

herramientas.

También es aprender a:

- Pensar con orden;
- Comunicar con responsabilidad;
- Verificar información;
- Organizar procesos;
- Cuidar datos;
- Diseñar con claridad;
- Construir con propósito.

Eso es lo que le da verdadero valor al programa.

Lo que debes llevarte de esta semana

Al finalizar esta semana debes comprender que:
una plataforma útil necesita memoria organizada;
CRUD ayuda a pensar operaciones sobre datos;
autenticación y autorización no son lo mismo;
el control de acceso protege información y orden;
la inclusión digital exige claridad y sencillez;
el proyecto final debe demostrar sentido, no solo técnica.

Tip de la semana

Para el proyecto final, piensa primero en el problema que resuelves, luego en la persona que lo usará y solo después en la herramienta que construirás.

Retos frecuentes y cómo resolverlos

Crear que el proyecto final vale más si es más complicado.

No.

Vale más si demuestra comprensión, orden, utilidad y sentido social.

Parte 7

Guías prácticas complementarias

32. Cómo crear tu primer repositorio local y publicar cambios

Paso 1. Entrar a tu carpeta de trabajo

```
cd nombre-de-tu-carpeta
```

Paso 2. Iniciar Git

```
git init
```

Paso 3. Agregar archivos

```
git add .
```

Paso 4. Crear un commit

```
git commit -m "Primer avance"
```

Paso 5. Cambiar el nombre de la rama principal

```
git branch -M main
```

Paso 6. Conectar con GitHub

```
git remote add origin URL_DEL_REPOSITORIO
```

Paso 7. Publicar

```
git push -u origin main
```



Qué debes revisar

- Que los archivos estén en la carpeta correcta;
- Que el commit tenga un mensaje claro;
- Que el repositorio remoto exista;
- Que estés usando la URL correcta.

33. Cómo abrir una carpeta del proyecto en Cursor

1. Abre Cursor.
2. Busca la opción Open Folder.
3. Selecciona la carpeta de tu proyecto.
4. Revisa el panel lateral.
5. Crea o abre un archivo.
6. Guarda cambios.
7. Si lo necesitas, abre la terminal integrada.

Recomendación

Pon nombres claros a tus archivos. Por ejemplo:

- Index.html
- Styles.css
- Server.js
- README.md

34. Cómo crear una página HTML básica

Paso 1. Crear archivo

Crea un archivo llamado index.html.

Paso 2. Escribir estructura mínima

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi página</title>
</head>
<body>
  <h1>Mi primer proyecto</h1>
  <p>Bienvenidos a esta página.</p>
</body>
</html>
```

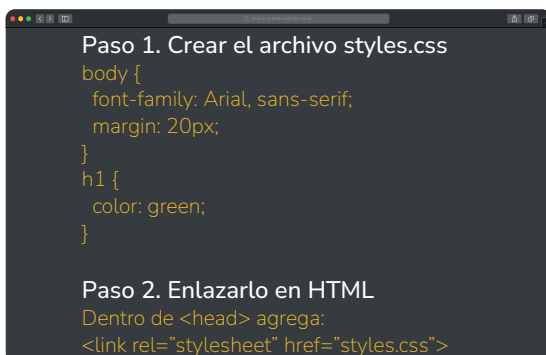
Paso 3. Guardar

Guarda el archivo.

Paso 4. Abrir en navegador

Haz doble clic sobre el archivo o usa una vista previa según tu entorno.

35. Cómo crear un archivo CSS y conectarlo



Resultado esperado

Tu página debe cambiar visualmente cuando guardes y recargues.

36. Cómo crear y ejecutar un servidor básico con Express

Paso 1. Crear carpeta

```
mkdir servidor-basico
```

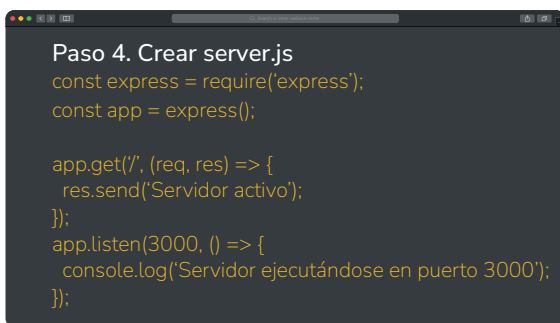
```
cd servidor-basico
```

Paso 2. Inicializar Node.js

```
npm init -y
```

Paso 3. Instalar Express

```
npm install express
```



Paso 5. Ejecutar

```
node server.js
```

Paso 6. Probar

Abrir en el navegador:

```
http://localhost:3000
```

37. Cómo crear rutas GET y POST simples

```
GET simple
app.get('/saludo', (req, res) => {
  res.send("Hola comunidad");
});

GET con parámetro
app.get('/mensaje/:nombre', (req, res) => {
  const nombre = req.params.nombre;
  res.send("Hola " + nombre);
});

Habilitar recepción JSON
app.use(express.json());

POST simple
app.post('/reporte', (req, res) => {
  const mensaje = req.body.mensaje;
  res.json({
    estado: 'Reporte recibido',
    mensaje: mensaje
  });
});
```

Qué debes entender

- GET consulta.
- POST envía o registra.
- Req contiene la solicitud.
- Res es la respuesta.

38. Cómo pensar una automatización simple con n8n

No siempre es necesario escribir mucho código para automatizar una tarea sencilla.

Ejemplo simple

Evento: entra un nuevo reporte.

Flujo: 1. detectar el nuevo reporte; 2. revisar que no tenga datos sensibles; 3. generar un mensaje corto; 4. enviarlo a un canal de seguimiento; 5. registrar que la notificación se envió.

Qué sí automatizar

- Notificaciones generales;
- Avisos de seguimiento;
- Actualizaciones de estado no sensibles;
- Tareas repetitivas.

Qué no automatizar sin controles

- Datos personales sensibles;
- Credenciales;
- Información privada sin protección.

39. Cómo pensar una historia de usuario

Una historia de usuario es una forma breve de expresar qué necesita una persona y para qué lo necesita.

Estructura simple

Como [tipo de usuario] quiero [acción] para [beneficio o propósito].

Ejemplos

- Como ciudadano quiero registrar una incidencia para dejar evidencia del caso.
- Como coordinador quiero recibir una alerta para revisar un nuevo reporte.
- Como miembro del equipo quiero consultar reportes para hacer seguimiento.

Parte 8

Seguridad, privacidad y buenas prácticas

40. Reglas mínimas de seguridad durante el curso

1. No publiques credenciales

No subas contraseñas, tokens o claves al repositorio.

2. No compartas datos sensibles sin protección

Si una práctica incluye información personal, revisa muy bien qué debe mostrarse y qué no.

3. Minimiza datos

Pide solo la información que realmente necesitas.

4. Usa mensajes claros

Cuando una persona use un formulario o una aplicación, debe entender qué está ocurriendo.

5. Registra cambios con orden

Git ayuda también a la trazabilidad y al control.

6. Revisa antes de publicar



No publiques
contenido o código
sin una revisión mínima.

41. Accesibilidad básica explicada fácil

Accesibilidad significa hacer que una herramienta sea más comprensible y usable para más personas.

Buenas prácticas mínimas

- Títulos claros;
- Estructura ordenada;
- Labels en formularios;
- Mensajes simples;
- Contraste y tamaño legible;
- Evitar pedir pasos confusos.

La accesibilidad no es un lujo. Es una parte importante de la inclusión digital.

42. Verificación responsable de información

Antes de compartir una información, pregúntate:

- ¿Quién la publicó?
- ¿Cuándo la publicó?
- ¿Existe otra fuente que la confirme?
- ¿Podría hacer daño si está equivocada?
- ¿La estoy entendiendo bien o solo vi el título?

Regla práctica

No compartas primero y revises después. Hazlo al revés.

Parte 9

Evaluación y seguimiento

43. Cómo se evalúa el proceso

La evaluación del programa es continua. Esto significa que el aprendizaje se revisa durante las semanas y también al cierre de cada módulo.

Componentes principales

Quiz semanales

Revisan comprensión progresiva de los contenidos.

Examen final del módulo

Revisa la integración de lo aprendido en ese módulo.

Evidencias técnicas y reflexivas

Ayudan a mostrar progreso real, aunque la evaluación formal se concentra en el LMS según la organización del curso.

Idea importante

La evaluación no busca castigarte.
Busca identificar avances,
errores frecuentes y temas
que necesitan refuerzo.



44. Qué se espera de tu proceso como estudiante

Se espera que puedas:

- Participar con respeto;
- Practicar de forma constante;
- Registrar avances;
- Pedir ayuda cuando algo no salga;
- Usar las herramientas con criterio;
- Relacionar lo técnico con su propósito social;
- Mejorar con la retroalimentación.

Parte 10

Glosario básico del estudiante

45. Glosario

Algoritmo

Lista ordenada de pasos para resolver un problema.

API

Forma organizada de comunicación entre sistemas o aplicaciones.

Autenticación

Proceso para verificar quién eres.

Backend

Parte interna de una aplicación que procesa información.

Base de datos

Lugar estructurado para guardar información.

Commit

Registro de una versión del proyecto en Git.

CSS

Lenguaje de estilos para páginas web.



Frontend

Parte visual de una aplicación.

Git

Herramienta para controlar versiones de un proyecto.

GitHub

Plataforma para alojar y compartir repositorios.

HTML

Lenguaje para estructurar páginas web.

JSON

Formato de texto para organizar datos.

Node.js

Entorno para ejecutar JavaScript fuera del navegador.

npm

Administrador de paquetes de Node.js.

Puerto

Número que ayuda a identificar por dónde se conecta una aplicación o servicio.

Repositorio

Carpeta o proyecto versionado con Git.

Ruta

Dirección que define cómo responde el servidor.

Servidor

Programa o sistema que responde solicitudes.

Terminal

Herramienta para ejecutar instrucciones escritas.

Trazabilidad

Posibilidad de seguir y revisar el historial de cambios o acciones.

Parte 11

Recordatorios finales para el estudiante

46. Diez ideas para avanzar bien en el programa

1. Empieza con calma.
2. No te compares con otros.
3. Practica aunque sea con ejercicios pequeños.
4. Guarda y ordena tus archivos.
5. Usa Git desde etapas tempranas.
6. Haz preguntas cuando algo no se entienda.
7. Verifica lo que te entregue una IA.
8. No publiques datos sensibles.
9. Relaciona cada práctica con su sentido social.
10. Confía en que puedes aprender paso a paso.

47. Cierre

Esta cartilla está pensada para acompañarte en un recorrido que une tecnología, comunicación, comunidad y participación democrática. Aprenderás herramientas técnicas, pero también criterios para usarlas con responsabilidad.

El objetivo no es solamente que sepas abrir un editor, crear un archivo HTML o levantar un servidor con Node.js. El objetivo es que entiendas que detrás de cada herramienta hay decisiones, impactos y oportunidades para construir soluciones útiles para las personas y las comunidades.



Aprender desde cero es posible. Lo más importante es mantener el orden, practicar con paciencia y avanzar una semana a la vez.

48. Anexo. Resumen ultra rápido por herramienta

Git

Sirve para registrar cambios.

GitHub

Sirve para publicar tu repositorio.

Cursor

Sirve para abrir carpetas, editar archivos y trabajar el proyecto.

HTML

Sirve para estructurar la página.

CSS

Sirve para dar estilo visual.

Node.js

Sirve para ejecutar JavaScript del lado del servidor.

Express

Sirve para crear rutas y servidores más fácilmente.

n8n

Sirve para automatizar tareas no sensibles.

Telegram

Puede servir como canal de coordinación o notificación comunitaria.

49. Anexo. Mini guía de estudio por semana

Antes de cada semana

- Revisa el objetivo;
- Identifica conceptos nuevos;
- Organiza tu carpeta;
- Prepara tu entorno.

Durante la semana

- Toma notas breves;
- Haz la práctica paso a paso;
- Guarda avances;
- Usa Git.

Al final de la semana

- Verifica archivos;

- Revisa errores;
- Completa la reflexión o evidencia ciudadana;
- Publica avances en el repositorio.

50. Anexo. Modelo simple de bitácora del estudiante

Semana:

Qué aprendí:

Qué práctica realicé:

Qué error tuve:

Cómo lo resolví:

Qué debo reforzar:

Qué relación encontré entre tecnología y comunidad:

Créditos

Presidente de la República de Colombia:

Doctor Gustavo Petro Urrego

Ministro de Igualdad y Equidad:

Doctor Luis Alfredo Acosta Zapata

Viceministro de Juventudes:

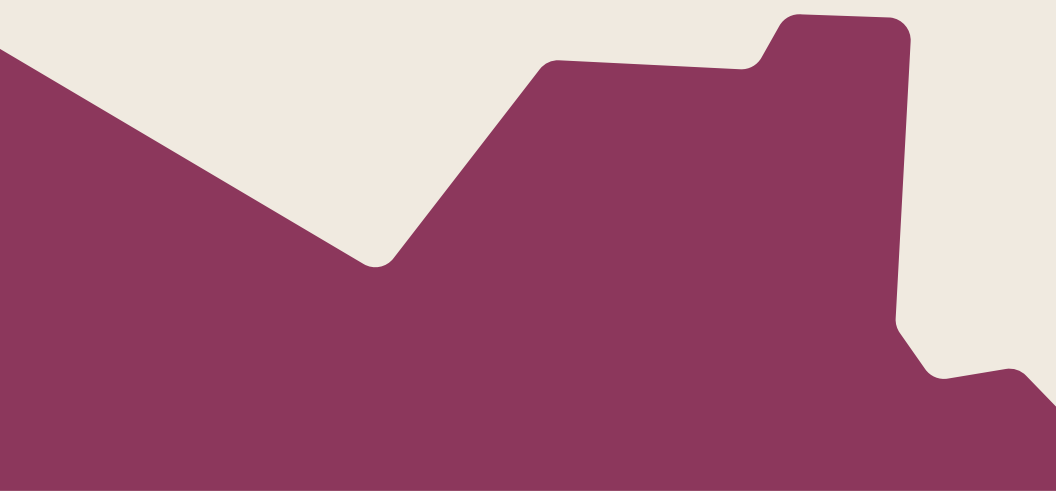
Doctor Pablo Mateo Zabala

Rector Universidad de Cartagena:

Doctor William Malkún Castillejo

Directora Centro de Transferencia, Innovación y Emprendimiento:

Doctora Marisol Benavides Villota





Universidad
de Cartagena
Fundada en 1827



Aprenderás
a **PROGRAMAR**
para **CONSTRUIR**
un **MUNDO**
más justo

